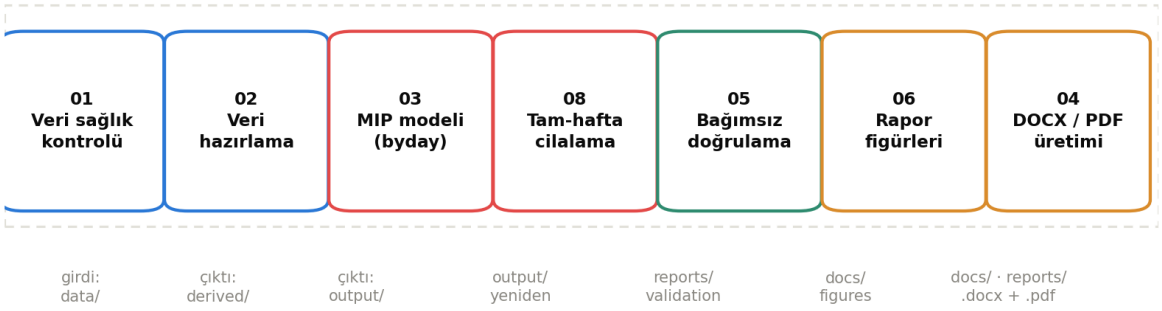
TEKNOFEST · Yapay Zekâ Destekli Havayolu Optimizasyonu

ALGORİTMA

sıfırdan, adım adım — jüri sunumu hazırlığı

Takım: Büke

run_all.py — tek komutla uçtan uca koşu



Bir adım hata verirse akış AÇIK hata ile durur — doğrulanmamış çözüm teslim edilmez.

Bu doküman üç soruya cevap vermek için yazıldı

1) "E2 ne, anlamadım" → Bölüm 6. **2)** "MIP gap %1,18 kesin mi, nereden biliyoruz?" → Bölüm 9. **3)** "100.552 ikili değişken ne demek?" → Bölüm 4 ve Ek A.
Ayrıca projenin tamamı — ham veriden nihai sonuca — baştan sona anlatılıyor.

Tüm sayılar kaynak koddan ve koşu raporlarından doğrulanmıştır; ikili değişken dökümü yeniden sayılarak teyit edilmiştir.

İçindekiler

	Bölüm	İçerik
0	Bu doküman ne işe yarar?	60 saniyelik özet · okuma sırası
1	Problem — neye karar veriyoruz?	aktarma ağı · J = E1 + boşluk + E2 · rekabetçilik
2	Veri — dört Excel dosyasından modele	sağlık kontrolü · sarma tuzağı · ρ · ayarlanabilir/sabit
3	Optimizasyon 101 — sıfırdan, jargonsuz	ikili değişken · LP gevşetmesi · dallanma-sınırlama · incumbent/bound
4	Modelin değişkenleri — yüz binin dökümü	x · y · z · w · β · γ · ön-işleme
5	Kısıtlar — kurallar ve gerekçeleri	A1–G · büyük-M · B2 çatışması · ceza terimi
6	**E2 — sorduğunuz sorunun tam cevabı**	**iki farklı E2 · min'in doğrusallaştırılması · neden kapalı**
7	Amaç fonksiyonu	iki ödül, bir ceza · hangi sayıyı raporluyoruz
8	Çözüm hattı	güne göre ayrıştırma · sıcak başlangıç · çift bariyer
9	**MIP gap — "%1,18 kesin mi?"**	**matematiksel kaynak · kodun 941. satırı · neyi garanti eder, neyi etmez**
10	Bağımsız doğrulama	çift kayıt · yakalanan üç gerçek hata
11	Sonuçlar	ana tablo · gün kırılımı · duyarlılık
12	Jüri soru bankası	10 soru, hazır cevaplar
A	Ek A — Sayı künyesi ve sayım kanıtı	iki modelin karşılaştırması · dosya haritası · bilinen tutarsızlıklar

0 Bu doküman ne işe yarar?

Salı günü jüri karşısında olacaksınız. Bu doküman, projenin **en başından** — yani elinize gelen dört Excel dosyasından — **en sonuna** — yani "çözümümüz optimalin en fazla %1,18 uzağında" cümlesine kadar her adımı, hiçbir adımı atlamadan anlatır.

Üç sorunuz vardı. Cevapları burada, ama nerede olduklarını baştan söyleyeyim:

Sorunuz	Cevabın yeri	Tek cümlelik cevap
"**E2** ne? Anlamadım."	**Bölüm 6** (tamamı)	Projede **iki ayrı E2** var ve karıştırılıyor: biri veri sütunu, biri model kısıtı. İkisi de Bölüm 6'da.
"**%1,18** kesin mi, nereden biliyoruz?"	**Bölüm 9** (tamamı)	Kesin. Çünkü bu sayı bizim iddiamız değil, çözücünün **matematiksel kanıtı**; kodun hangi satırında okunduğunu da göstereceğim.
"**100.552 ikili değişken** ne demek?"	**Bölüm 4** + **Ek A**	Modelin sorduğu evet/hayır sorularının sayısı. Dökümünü **yeniden sayarak** doğruladım — ve raporlardaki iki farklı sayının nedenini buldum.

> **Okuma sırası önerisi**
>
> Zamanınız kısıtlıysa: **Bölüm 3** (optimizasyon temelleri) → **Bölüm 6** (E2) → **Bölüm 9** (MIP gap) → **Bölüm 12** (jüri soruları). Bu dördü, sorduğunuz soruların tamamını kapatır. Bölüm 1–2 problemi ve veriyi, 4–5 modeli, 7–8 çözüm yöntemini, 10–11 doğrulama ve sonuçları anlatır.

0.1 Önce 60 saniyelik özet — tüm proje tek sayfada

Jüri size "projenizi bir dakikada anlatın" derse söyleyeceğiniz şey budur:

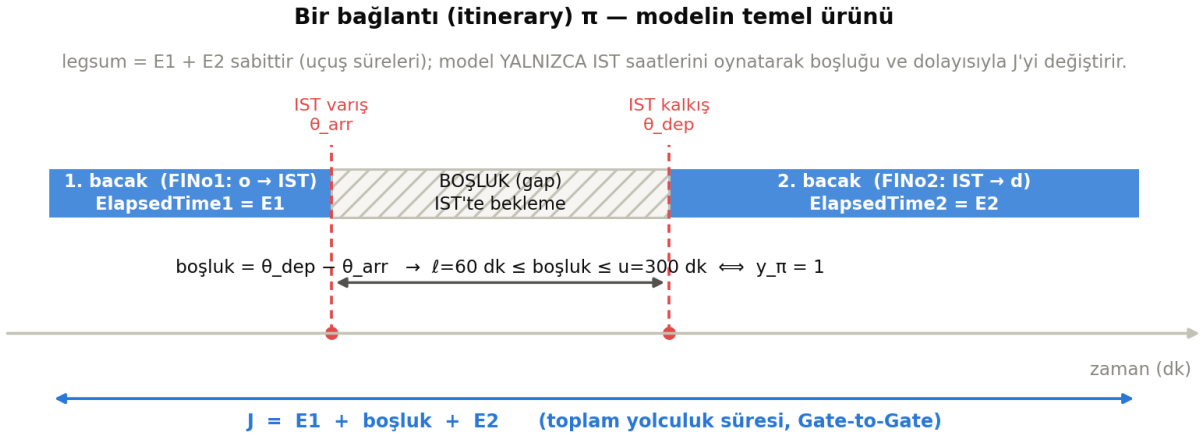
1. **Problem.** THY'nin İstanbul aktarma merkezinde (IST) haftalık bir uçuş tarifesi var. Bir yolcu Berlin'den Bangkok'a giderken IST'te aktarma yapıyor. Bu yolculuğun toplam süresi, rakip havayollarının aynı rotadaki süreleriyle yarışıyor. **Uçuş saatlerini ±60 dakika kaydırarak** bu yarışta daha iyi bir yere gelebilir miyiz?
2. **Karar.** Değiştirdiğimiz tek şey: ayarlanabilir uçuşların IST'teki kalkış/varış saatleri. Uçak filosunu, rotaları, uçuş sürelerini, hiçbirini değiştirmiyoruz.
3. **Ölçüt.** Her pazarın (Berlin→Bangkok gibi) bir gelir ağırlığı ρ var. Fayda = ρ × (kaç geçerli bağlantı sunduğumuz + rakip sıralamasında ne kadar yükseldiğimiz). Tüm pazarlar üzerinden toplanır.
4. **Yöntem.** Problemi bir **Karışık Tamsayılı Programlama (MIP)** modeline çevirdik: yüz binin üzerinde evet/hayır değişkeni. Sezgisel (genetik algoritma vb.) kullanmadık — bilinçli bir tercih, nedeni Bölüm 9'da.
5. **Çözüm.** Açık kaynak HiGHS çözücüsü, iki aşamada: önce gün gün (hızlı), sonra tüm hafta tek modelde cilalama (30 dakika).
6. **Sonuç.** Rekabetçilik **+%4,72**. Bunun için uçuşların yalnız **%23'ü**, ortalama **19 dakika** kaydırıldı. Ve çözümün optimale en fazla **%1,18** uzak olduğu **kanıtlandı**.
7. **Güvence.** Sonuç, modeli hiç görmeyen **bağımsız bir doğrulayıcıyla** baştan hesaplandı: kontrollerin tamamı PASS.

1 Problem — neye karar veriyoruz?

Optimizasyon modelinin ilk kuralı şudur: **neyin karar, neyin veri olduğunu** netleştirmeden tek satır kod yazamazsınız. Bu bölüm tam olarak onu yapıyor.

1.1 Aktarma ağı ve "bağlantı" kavramı

THY'nin ağı bir **hub-and-spoke** (merkez-ve-parmak) ağıdır. Yolcuların büyük bölümü A noktasından B'ye doğrudan uçmaz; IST'e gelir, uçak değiştirir, oradan devam eder. Bu iki bacaklı yolculuğa **bağlantı** (connection / itinerary) diyoruz.



Şekil 1.1 — Bir bağlantının anatomisi. Modelin oynadığı tek şey ortadaki boşluktur; iki bacağın uçuş süreleri sabittir.

Bir bağlantının üç parçası vardır ve bunları ayırt etmek **kritik** — çünkü sorduğunuz "E2" sorusunun cevabı burada başlıyor:

Parça	Sembol	Ne demek	Karar mı, veri mi?
1. bacak süresi	**E1** *(ElapsedTime1)*	Örn. Berlin→IST uçuşunun havada geçen süresi	**VERİ** — değişmez
Aktarma boşluğu	**boşluk** *(gap)*	Uçağın IST'e varışı ile sonraki uçağın kalkışı arasındaki bekleme	**KARAR** — saatleri kaydırınca değişir
2. bacak süresi	**E2** *(ElapsedTime2)*	Örn. IST→Bangkok uçuşunun havada geçen süresi	**VERİ** — değişmez

> **İlk "E2" burada — ve bu bir KISIT DEĞİL**
>
> Verideki `ElapsedTime2` sütunu kodda `E2_min` adını alır ve **ikinci bacağın uçuş süresidir**. Bunun modelin "E2 kısıtı" ile **hiçbir ilgisi yoktur**. Aynı harfin iki farklı şeye verilmiş olması talihsiz bir isim çakışmasıdır; Bölüm 6.1'de ikisini yan yana koyup ayıracağım.

Toplam yolculuk süresi (J) bu üçünün toplamıdır:

$$J = E1 + \text{boşluk} + E2$$

E1 ve E2 sabit olduğuna göre, **J'yi değiştirmenin tek yolu boşluğu değiştirmektir** — yani uçuş saatlerini kaydırmak. Projenin tüm mantığı bu tek denklemin üzerine kurulu.

1.2 Neyi değiştiriyoruz, neyi değiştirmiyoruz



Şekil 1.2 — Kararımız tek bir şey: ayarlanabilir uçuşların IST saatleri.

DEĞİŞTİRDİĞİMİZ (karar)	DEĞİŞTİRMEDİĞİMİZ (veri / dış kısıt)
Ayarlanabilir uçuşların IST kalkış/varış saati	Uçuş süreleri (bacak blokları)
— 10 dakikalık ızgarada	Hangi uçuş nereye uçuyor (rotalar)
— mevcut saatin ±60 dakika penceresinde	Filo, uçak sayısı, kapasite
Hangi bağlantının "sunulduğu" (bunun sonucu olarak)	Rakiplerin tarifeleri
	Pazarların gelir ağırlıkları ρ

Neden ±60 dakika? Brief bir sınır koymuyor, ama sınırsız bırakmak iki sorun yaratırdı: *(i)* modelin boyutu patlar (her uçuş için 13 seçenek yerine 144), *(ii)* gerçek hayatta bir uçuşu 5 saat kaydırmak operasyonel olarak anlamsızdır. ±60 dk bilinçli ve belgeli bir tasarım kararıdır (`docs/model_formulation.md`, karar D3).

Neden 10 dakikalık ızgara? Çünkü brief'in hub kapasite kısıtı zaten 10 dakikalık kovalarda tanımlı. Aynı ızgarayı kullanmak hem kısıtı doğal kılar hem modeli küçültür. **±60 dk penceresi ÷ 10 dk ızgara = uçuş başına 13 seçenek** (−60, −50, …, 0, …, +50, +60). Bu "13" sayısını aklınızda tutun; Bölüm 4'te yüz binlik değişken sayısının nereden geldiğini açıklarken karşınıza çıkacak.

1.3 Neyi iyileştiriyoruz — "rekabetçilik" tam olarak ne?

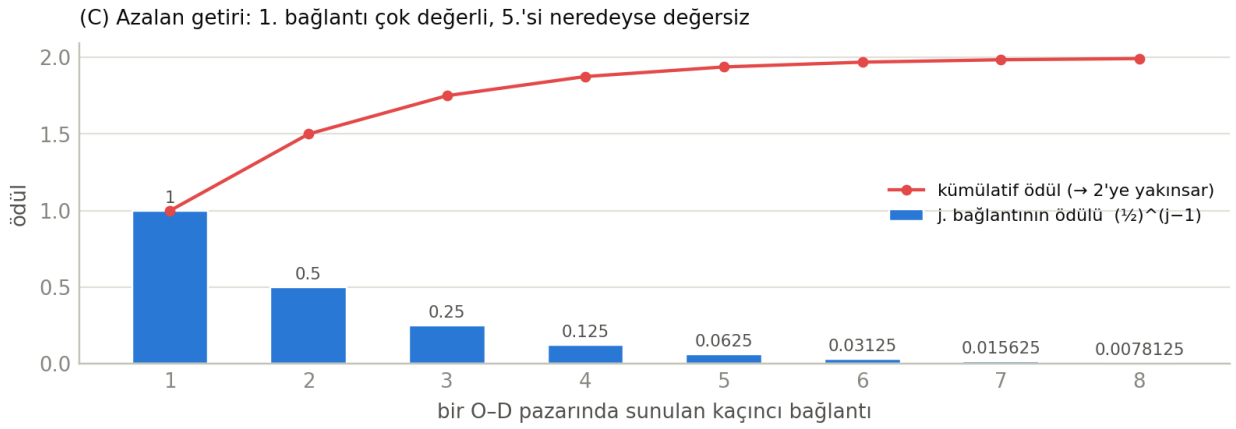
Fayda iki bileşenden oluşur. Her ikisi de her (pazar, gün) için ayrı hesaplanır ve o pazarın gelir ağırlığı ρ ile çarpılır.

(a) Bağlantı sayısı ödülü — azalan getiri

Bir pazarda ne kadar çok geçerli aktarma sunarsanız o kadar iyi. Ama ikinci bağlantı birinciden, üçüncü ikinciden daha az değerlidir (yolcuya zaten bir seçenek sunmuşsunuzdur). Bunu **yarıya inen ağırlıklarla** modelledik:

1. bağlantı: 1,000 · 2.: 0,500 · 3.: 0,250 · 4.: 0,125 …

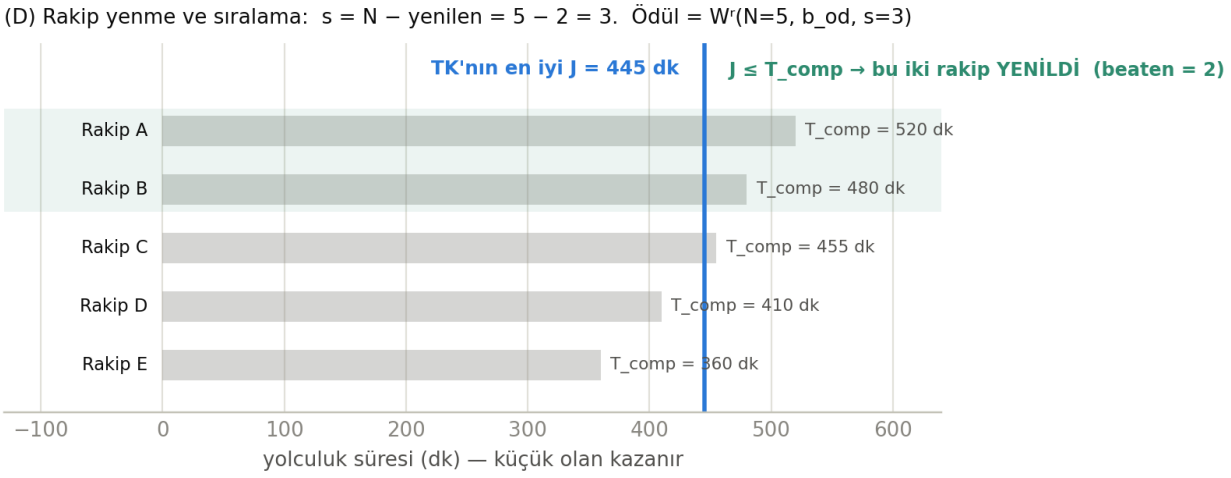
yani j. bağlantının ağırlığı **(½)^(j−1)**



Şekil 1.3 — Azalan getiri: her ek bağlantının katkısı yarıya iner.

(b) Sıralama ödülü — rakibi yenmek

Bir pazarda N rakip taşıyıcı var. Bizim en iyi bağlantımızın toplam süresi J, bir rakibin süresinden **kısaysa** o rakibi "yendik" sayılır. Yenilen rakip sayısı sıralamamızı belirler, sıralama da yarışmanın verdiği **ödül tablosundan** (`change_ranking_input.xlsx`) okunur.



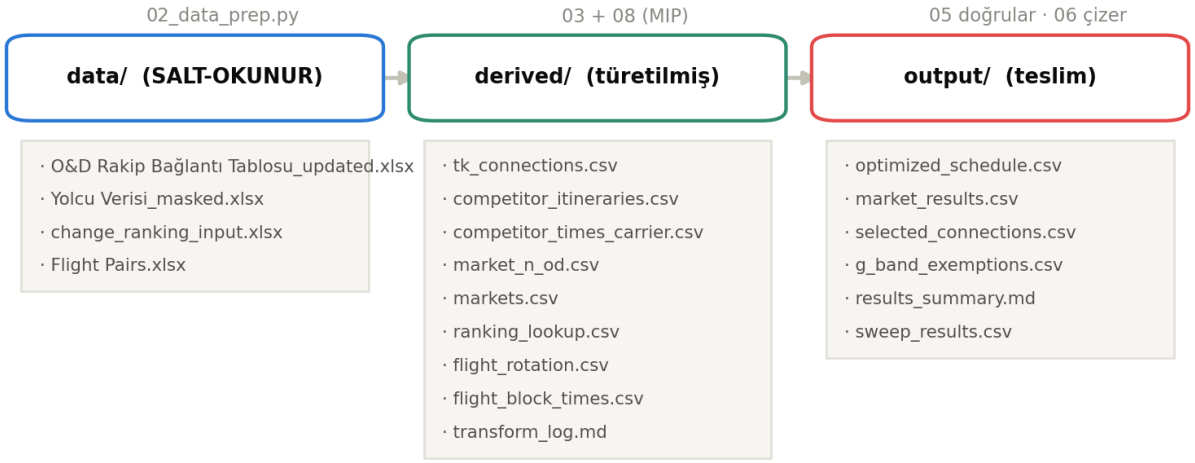
Şekil 1.4 — Yenilen rakip sayısı → yeni sıra → tablodan okunan ödül.

Neden bu ikisi birden?

Yalnız bağlantı sayısını ödüllendirseydik model, hiçbir rakibi yenmeyen ama çok sayıda vasat bağlantı üretirdi. Yalnız sıralamayı ödüllendirseydik tek bir çok hızlı bağlantı yapıp gerisini boşlardı. İkisi birlikte, brief'in istediği dengeyi kurar: **hem çok, hem hızlı**.

2 Veri — dört Excel dosyasından modele giden yol

Model kurmadan önce veriyi anlamak ve temizlemek zorundayız. Bu iş iki betikte yapılır: `01_data_health_check.py` (teşhis — hiçbir şeyi düzeltmez, sadece raporlar) ve `02_data_prep.py` (dönüşüm — `derived/` katmanını üretir).



Altın kural: yarışmanın orijinal Excel dosyalarına ASLA yazılmaz; her dönüşüm derived/transform_log.md içine kaydedilir.

Şekil 2.1 — Ham Excel → derived/ → model. Ham dosyalara hiç dokunulmaz.

2.1 Elimizdeki dört dosya

Dosya	Satır	Ne içerir
`O&D Rakip Bağlantı Tablosu_updated.xlsx`	**57.317**	Her satır bir aktarmalı yolculuk. 7.850'si THY, 49.467'si rakip. Sütunlar: iki bacağın uçuş no'ları, saatleri, süreleri (`ElapsedTime1/2`), taşıyıcı, pazar (O&D), gün.
`Yolcu Verisi_masked.xlsx`	8.075	Pazar başına **gelir/önem ağırlığı ρ** (1–1000 arası). Amaç fonksiyonunun ağırlıkları buradan gelir.
`change_ranking_input.xlsx`	22.140	**Sıralama ödül tablosu**: (rakip sayısı N, ilk sıra b, son sıra s) → ödül. Her N için tam N^2 satır.
`Flight Pairs.xlsx`	1.562	Uçak **rotasyon eşleri**: hangi gidiş uçuşu hangi dönüşle aynı uçağı kullanıyor. A2 kısıtının kaynağı.

Toplam **89.094 satır** ham veri. Modelin ihtiyaç duyduğu her şey bunlardan türetilir.

2.2 Veri sağlık kontrolü — ne bulduk?

`01_data_health_check.py` hiçbir şeyi düzeltmez; sadece bakar ve raporlar (`reports/data_health_report.md`). Bulduğu ve modeli şekillendiren başlıca şeyler:

Bulgu	Sayı	Ne yaptık
ρ değeri olmayan pazar	**48**	Amaç fonksiyonundan **dışladık**. Değer uydurmak yerine ihtiyatlı dışlama.

Bulgu	Sayı	Ne yaptık
Rotasyon eşi olmayan TK uçuş no	**176**	**Sabit** kabul ettik — hiç kaydırmıyoruz (rotasyon güvenliği doğrulanamaz).
Aynı (uçuş, gün) için farklı saat	**132**	Hafta normalizasyonu ile düzelttik (bkz. 2.4).
Rakip sayısı N (taşıyıcı bazlı)	1–17	Ödül tablosu N ≤ 40'ı kapsıyor → **tam kapsam**. Itinerary bazlı sayımda 73 pazar 40'ı aşıp tablo dışına çıkardı → taşıyıcı bazlı tanım seçildi.
Bağlantı boşluğu 60–300 dk penceresinde	%89,0	Kalan %11 zaten geçersiz; ön-işlemede elenir.

2.3 J'nin kesin hesabı — ve "sarma" tuzağı

Veri toplam yolculuk süresini `Gate-to-Gate Uçuş Süresi` sütununda veriyor. Ama bu sütun **saat formatında** (time-of-day): 25 saatlik bir yolculuk 01:00 olarak görünüyor. Buna **sarma** (wrap) denir ve düzeltilmezse ultra-uzun uçuşlarda tam 24 saatlik hata yapar.

Neyse ki güncellenmiş veri iki bacağın sürelerini ayrı ayrı veriyor (`ElapsedTime1`, `ElapsedTime2`). O zaman J'yi **doğrudan** kurabiliriz:

`src/02_data_prep.py` · satır 137–155 · J = E1 + boşluk + E2 — sarma belirsizliği YOK

```
137     has_elapsed = {"ElapsedTime1", "ElapsedTime2"}.issubset(conn.columns)
138     if has_elapsed:
139         conn["E1_min"] = conn["ElapsedTime1"].map(dur_min)
140         conn["E2_min"] = conn["ElapsedTime2"].map(dur_min)
141         n_bad = int(conn["E1_min"].isna().sum()) + int(conn["E2_min"].isna().sum())
142         if n_bad:
143             raise ValueError(f"[ŞEMA] ElapsedTime1/2 çözümlenemeyen hücre: {n_bad} "
144                              f"(örnek: {conn.loc[conn['E1_min'].isna(), 'ElapsedTime1'].head(3).tolist()})")
145         conn["legsum_min"] = conn["E1_min"] + conn["E2_min"]
146         conn["J_min"] = conn["legsum_min"] + conn["gap_min"]
147         conn["dewrap_k"] = np.round((conn["J_min"] - conn["J_wrap_min"]) / 1440.0).astype(int)
148 
149         # doğrulama: kesin J, sarılı Gate-to-Gate ile mod-24sa tutarlı olmalı
150         mod_off = ((conn["J_min"] - conn["J_wrap_min"]) % 1440.0)
151         inconsistent = ~(np.isclose(mod_off, 0, atol=1.01) | np.isclose(mod_off, 1440, atol=1.01))
152         log(f"\n[J KESİN] ElapsedTime1/2 mevcut -> J = E1 + boşluk + E2 (wrap belirsizliği YOK)")
153         log(f"  Gate-to-Gate ile mod-24sa tutarsız satır: {int(inconsistent.sum())} (0 beklenir)")
154         log(f"  24 saati aşan gerçek yolculuk: {int((conn['J_min'] >= 1440).sum())} satır"
155             f" (eski şemada bunlar sarılı görünüyordu)")
```

Satır 145–146 tam olarak Bölüm 1.1'deki denklemdir. Satır 150–151 ise bir **öz-doğrulamadır**: hesapladığımız kesin J ile verideki sarılı sütun, 24 saatin katları dışında birbirini tutmalı. Sonuç: **tutarsız satır 0**. Yani iki bağımsız kaynak birbirini doğruluyor.

> **Bu bir jüri kozu**
>
> Veride gerçekten 24 saati aşan **495** yolculuk var. Eski (ElapsedTime'sız) sezgisel yöntem bunların **154'ünü** yanlış hesaplıyordu. Yani bu düzeltme kozmetik değil: 154 satırda tam 24 saatlik hata engellendi. Jüri "veriyi ne kadar iyi anladınız" diye sorarsa cevap budur.

2.4 Hafta normalizasyonu — gerçek bir hatanın yakalanışı

Veride aynı operasyon günü ("Gün 1") farklı pazarlarda **farklı takvim haftalarına** yazılmış: bir pazarda 4 Ağustos Pazartesi, diğerinde 11 Ağustos Pazartesi. Model bunları **iki ayrı uçuş** sanar ve aynı fiziksel uçağa iki farklı saat atayabilir — çıktıda bu gerçekten gözlendi (bir kopya −50 dk, öteki 0).

`src/02_data_prep.py` · satır 198–212 · her iki zaman damgası BİRLİKTE 7k gün geri çekilir

```
198     ev_key = lambda df: set(zip(df["FlNo1"], df["day"], df["ist_arr"])) \
```

```
                      | set(zip(df["FlNo2"], df["day"], df["ist_dep"]))
    n_ev_before = len(ev_key(tk_out))
    dep_date = tk_out["ist_dep"].dt.normalize()
    canon = dep_date.groupby(tk_out["day"]).transform("min")
    k_days = (dep_date - canon).dt.days
    weekly = (k_days % 7 == 0)
    shift_d = np.where(weekly, k_days, 0)
    tk_out["ist_arr"] = tk_out["ist_arr"] - pd.to_timedelta(shift_d, unit="D")
    tk_out["ist_dep"] = tk_out["ist_dep"] - pd.to_timedelta(shift_d, unit="D")
    n_ev_after = len(ev_key(tk_out))
    log(f"\n[Hafta normalizasyonu] geri çekilen satır: {int((shift_d > 0).sum())} "
        f"(7'nin katı olmayan, dokunulmayan: {int((~weekly).sum())})")
    log(f"  Tekil (uçuş,gün,zaman) olay: {n_ev_before} -> {n_ev_after} "
        f"(birleşen hafta-kopyası: {n_ev_before - n_ev_after}; boşluk/J değişmedi)")
```

Kritik nokta satır 206–207: iki zaman damgası **birlikte** kaydırılır, dolayısıyla aralarındaki boşluk ve J **hiç değişmez**. Sonuç: 132 satır normalize edildi, olay sayısı 5.604 → 5.474'e indi. Ardından birebir özdeş adaylar tekilleştirildi (7.850 → 7.693) — aksi hâlde aynı fiziksel bağlantı **iki kez ödül** kazandırırdı.

2.5 ρ (gelir ağırlığı) nereden geliyor?

Bu soruyu jüri sorabilir; cevabı çok basit: **ρ hesaplanmıyor, doğrudan okunuyor.** `Yolcu Verisi_masked.xlsx`'teki `OD importance factor` sütunu, pazar anahtarıyla eşleştirilip aynen kullanılır. Hiçbir normalizasyon, ölçekleme veya dönüşüm yok — 1..1000 aralığındaki ham değer. Bu bilinçli: yarışmanın verdiği ağırlığı değiştirmek, yarışmanın ölçütünü değiştirmek olurdu.

2.6 Hangi uçuş "ayarlanabilir", hangisi "sabit"?

Bu ayrım modelin boyutunu doğrudan belirler, o yüzden net olalım:

`src/03_model.py` · satır 238–246 · sabit uçuşun tek adayı var: mevcut saati

```
        key = (int(flno), role, cur)
        if key not in events:
            adj = int(flno) not in fixed_flnos          # rotasyon partneri yoksa SABIT (docs D6)
            if adj:
                cands = [cur + k * delta for k in range(-(wd // delta), wd // delta + 1)]
            else:
                cands = [cur]
            events[key] = dict(id=len(events), flno=int(flno), role=role,
                               cur=cur, day=int(day), adjustable=adj, cands=cands)
```

Kural (satır 240): bir TK uçuş numarasının `Flight Pairs.xlsx`'te **rotasyon eşi yoksa sabittir**. Gerekçe: eşi bilinmeyen bir uçuşu kaydırırsak, o uçağın bir sonraki görevine yetişip yetişmediğini **doğrulayamayız**. Emin olamadığımız yerde oynamıyoruz.

	Uçuş no	Model olayı	Aday saat sayısı
Ayarlanabilir	776	**4.553**	13 (±60 dk / 10 dk)
Sabit	176	823	1 (mevcut saati)
Toplam	952	**5.376**	—

Not: "uçuş no" ile "olay" farklı şeylerdir. Bir uçuş numarası haftanın birden çok gününde uçar; her (uçuş no, rol, gün) üçlüsü ayrı bir olaydır ve ayrı karar verilir. 776 ayarlanabilir uçuş numarası → 4.553 ayarlanabilir olay.

3 Optimizasyon 101 — sıfırdan, jargonsuz

Bu bölüm, matematiksel programlamayı hiç bilmeyen birine anlatır gibi yazıldı. Jüri temel kavramları sorarsa buradaki cümleleri doğrudan kullanabilirsiniz.

3.1 Karar değişkeni, kısıt, amaç — üç parça

Her optimizasyon modeli üç şeyden oluşur:

Parça	Sorusu	Bizim projemizde
Karar değişkeni	Neyi seçebiliyorum?	Her ayarlanabilir uçuş hangi saate atansın?
Kısıt	Hangi seçimler yasak?	Uçak dönüşe yetişmeli, hub kapasitesi aşılmamalı, aynı uçuş günler arası ±15 dk'dan fazla oynamamalı…
Amaç fonksiyonu	"İyi" ne demek?	ρ-ağırlıklı rekabetçilik faydası — enbüyüklenecek.

3.2 Neden "ikili" (binary) değişken?

Bazı kararlar kesirli **olamaz**. "2,7 uçuş" diye bir şey yoktur. Daha önemlisi, bazı kararlar doğası gereği **EVET / HAYIR**'dır:

- Bu uçuş 14:30 kovasına mı atandı? **Evet (1) / Hayır (0)**
- Bu bağlantı sunuluyor mu? **Evet (1) / Hayır (0)**
- Bu rakip yenildi mi? **Evet (1) / Hayır (0)**

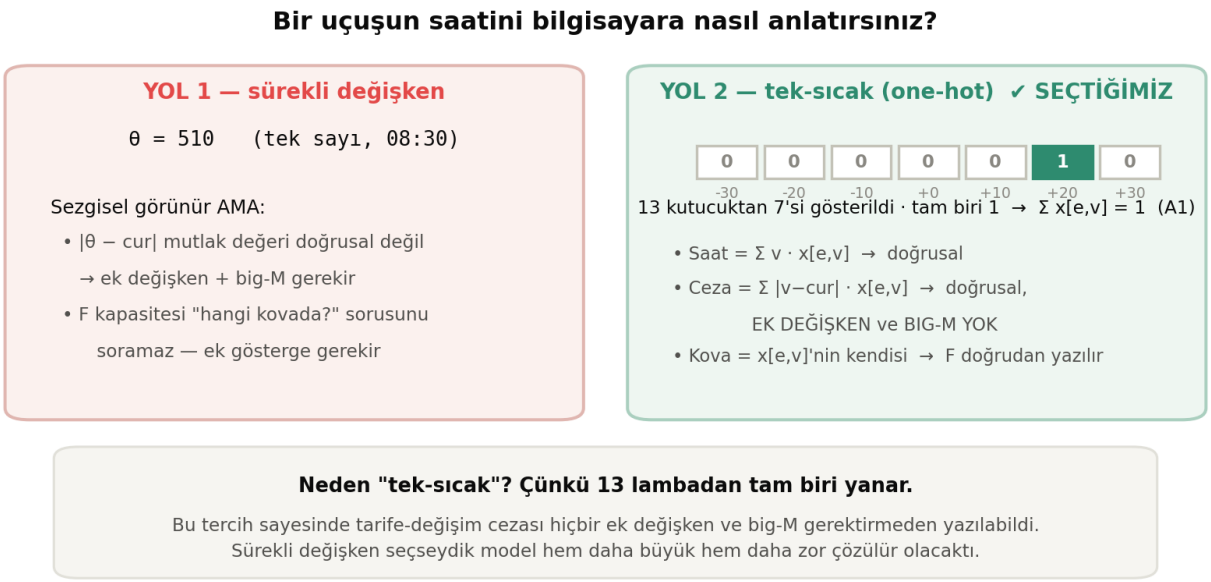
Yalnız 0 veya 1 değeri alabilen değişkene **ikili (binary) değişken** denir. Bunlar modelin en güçlü parçasıdır — ama aynı zamanda en pahalısı. Nedenini anlamak için önce çözücünün nasıl çalıştığını görmek gerek.

"Tek atama" kalıbı (one-hot) — en sık kullandığımız yapı

Bir uçuşun 13 aday saatinden **tam birine** atanmasını nasıl söylersiniz? 13 tane ikili değişken açıp toplamlarını 1'e eşitleyerek:

`src/03_model.py` · satır 372–375 · A1 — her ayarlanabilir olay tam bir kovaya atanır

```
372     # --- (A1) her ayarlanabilir olay tam bir kovaya atanır -----------------
373     for e in inst.ev_by_id.values():
374         if e["adjustable"]:
375             m.cons.add(sum(m.x[e["id"], v] for v in e["cands"]) == 1)
```



Şekil 3.1 — 13 ikili değişkenden tam biri 1 olur; uçuşun saati budur.

Bu yapının güzelliği şu: uçuşun gerçek saati artık **doğrusal bir ifadedir** — `θ = Σ (saat değeri × x)`. Toplamdaki 13 terimin 12'si sıfırla çarpılır, biri kalır. Böylece "saat" gibi bir sayıyı ikili değişkenlerle **tam doğrusal** biçimde modelleyebiliyoruz. Bu kalıp Bölüm 5'te ceza teriminin neden bedavaya geldiğini de açıklayacak.

3.3 LP gevşetmesi — çözücünün en önemli numarası

Yüz binlik ikili değişkenin tüm kombinasyonlarını denemek imkânsızdır. Çözücü bunun yerine şu numarayı yapar:

> **LP gevşetmesi (linear relaxation)**
> "x ya 0 ya 1 olmalı" kuralını **geçici olarak kaldır**; x'in 0 ile 1 arasında herhangi bir kesirli değer almasına izin ver. Ortaya çıkan problem artık bir **doğrusal programdır** (LP) ve LP'ler çok hızlı, kesin çözülür (simpleks / iç-nokta yöntemleri).

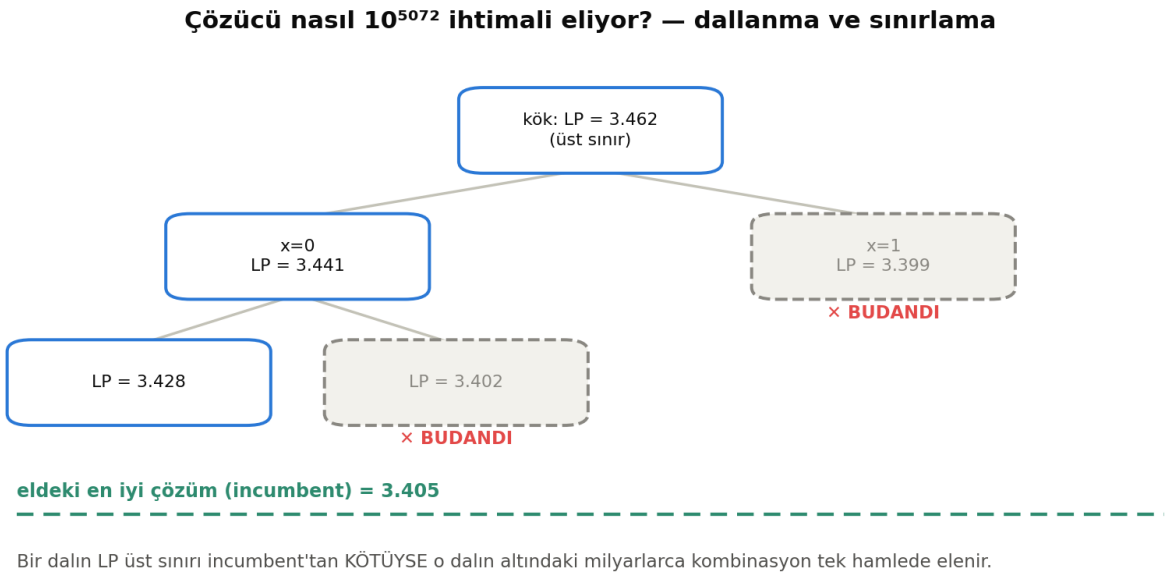
Peki bunun ne faydası var? Şu: gevşetilmiş problem, gerçek problemden **daha serbesttir** — daha fazla seçeneği vardır. Dolayısıyla bulduğu en iyi değer, gerçek problemin en iyi değerinden **asla kötü olamaz**. Enbüyükleme yaptığımız için bu şu demek:

LP'nin cevabı ≥ gerçek MIP'in cevabı

İşte bu bir **üst sınırdır** (bound): "cevap bundan daha iyi olamaz" garantisi. Bölüm 9'daki %1,18'in yarısı tam olarak bu cümleden geliyor.

3.4 Dallanma-sınırlama (branch and bound)

LP gevşetmesi genelde kesirli bir cevap verir ("bu uçuş %30 saat 14:30'da, %70 saat 14:40'ta"). Bu gerçek bir tarife değildir. Çözücü o zaman **dallanır**: kesirli çıkan bir değişkeni seçer ve iki alt problem kurar — birinde x=0, diğerinde x=1. Her alt problemi yine LP ile çözer. Ağaç böyle büyür.



Şekil 3.2 — Dallanma ve budama. Bir dalın LP üst sınırı eldeki en iyi çözümden kötüyse, o dalın altındaki tüm kombinasyonlar tek hamlede elenir.

Budama (pruning) işin sihridir: bir dalın LP üst sınırı 3,399 çıkmışsa ve elimizde zaten 3,405 değerinde gerçek bir çözüm varsa, o dalın altında hiçbir şey 3,399'u geçemez — dolayısıyla o dalın altındaki milyarlarca kombinasyon tek hamlede elenir. Çözücü devasa arama uzayını işte böyle yönetilebilir kılıyor.

Ve şimdi her ikili değişkenin neden **pahalı** olduğunu anlayabiliriz: her ikili değişken, dallanma ağacında potansiyel bir çatallanma noktasıdır. Değişken sayısı arttıkça ağaç büyür.

3.5 İki sayı: incumbent ve bound

Çözücü çalışırken sürekli **iki** sayı tutar. Bunları karıştırmamak, Bölüm 9'u anlamanın anahtarıdır:

	Adı	Ne demek	Enbüyüklemede rolü
Elde olan	**incumbent**	Şu ana kadar bulunmuş en iyi **GERÇEK** çözüm — tüm kısıtları sağlayan, tüm ikili değişkenleri gerçekten 0/1 olan bir tarife	**ALT** sınır — optimum bundan **küçük olamaz**
Kanıt	**bound**	LP gevşetmelerinden gelen "cevap bundan iyi olamaz" sınırı	**ÜST** sınır — optimum bundan **büyük olamaz**

Gerçek optimum bu iki sayının **arasında** bir yerdedir. Çözüm ilerledikçe incumbent yükselir, bound düşer; ikisi buluştuğunda (gap = 0) çözüm **kanıtlanmış optimaldir**.

$$\text{MIP gap} = |\,\text{bound} - \text{incumbent}\,| \;/\; |\,\text{incumbent}\,|$$

Şimdiden bir uyarı

Bizim gap'imiz %1,18 ve bu bir **belirsizlik** değil, bir **garantidir**: "bulduğumuz çözüm, ulaşılabilecek en iyisinden en fazla %1,18 uzakta." Ama bu garanti tam olarak neyin garantisi — ve neyin değil — Bölüm 9'da titizlikle ayrılıyor. Jüri oradan soru sorarsa hazırlıklı olmalısınız.

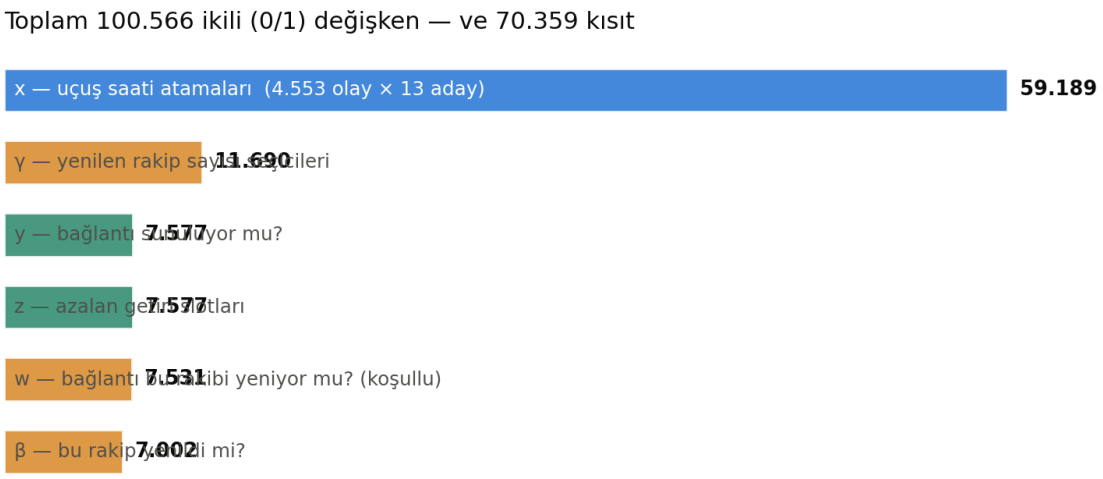
4 Modelin değişkenleri — yüz binin tam dökümü

Sorduğunuz sayı buydu. Şimdi tek tek nereden geldiğini göstereceğim — ve raporlarda neden **iki farklı sayı** (100.552 ve 100.566) göründüğünü de açıklayacağım. Bu bölümdeki tüm sayıları, modeli yeniden kurup **yeniden sayarak** doğruladım; sayım çıktısı Ek A'da.

4.1 Altı soru, altı değişken ailesi

Model aslında altı farklı soru sorar. Her soru bir ikili değişken ailesidir:

Değişken	Sorduğu soru	Adet	Nereden geliyor
`x[olay, saat]`	Bu uçuş olayı bu saate mi atandı?	**59.177**	4.553 ayarlanabilir olay × 13 aday saat (eksi sabitlenen 1 olay → −12)
`γ[pazar-gün, sayı]`	Bu pazarda tam kaç rakip yenildi? (sayı seçici)	**11.690**	Her pazar-gün için 0'dan N'e kadar seçenek → Σ (N+1)
`y[bağlantı]`	Bu bağlantı sunuluyor mu?	**7.577**	Aday bağlantı sayısı
`z[pazar-gün, slot]`	Azalan-getiri slotu dolu mu?	**7.577**	Her bağlantı için bir slot
`w[bağlantı, rakip]`	Bu bağlantı bu rakibi yeniyor mu?	**7.529**	Yalnız **koşullu** olanlar (bkz. 4.4)
`β[pazar-gün, rakip]`	Bu rakip yenildi mi?	**7.002**	Yenilebilir (pazar-gün, rakip) çiftleri
TOPLAM		**100.552**	nihai teslim modeli



Şekil 4.1 — Değişkenlerin yaklaşık %59'u uçuş saati atamalarıdır; gerisi ödül mantığını doğrusallaştıran yardımcı göstergelerdir.

Şekildeki toplam 100.566, tablodaki 100.552 — hangisi doğru?

İkisi de doğru; farklı modellerin sayıları. 100.566, modelin "monolitik" varyantıdır (`03_model.py full`). 100.552 ise **nihai teslim modelidir** (`08_fullweek_polish.py`): orada bir olay, gün-görünümü kopyası olduğu için aşama-1 değerine sabitlenir ve 12 `x` + 2 `w` değişkeni düşer. Yeniden-sayım kanıtı **Ek A**'da. Jüri sorarsa: "*Teslim ettiğimiz model 100.552; 100.566 aynı modelin bir olayı sabitlenmemiş halidir.*"

4.2 `x` — kalbin attığı yer

Değişkenlerin yaklaşık %59'u tek bir aileden geliyor ve mantığı çok basit: **4.553 ayarlanabilir olay × 13 aday saat**. Bölüm 1.2'de "13" sayısını aklınızda tutun demiştim — ±60 dakikalık pencere, 10 dakikalık ızgara, yani −60 / −50 / … / +60.

4.553 × 13 = 59.189. Nihai modelde bundan 12 eksik (59.177), çünkü bir olay sabitlenmiş durumda: 13 aday yerine 1.

Bu ailenin üzerine A1 kısıtı gelir — her olay için tam bir tanesi 1 olur (Bölüm 3.2'deki one-hot kalıbı). Modelin geri kalan her şeyi bu `x`'lerden **türetilir**: uçuşun saati, bağlantının boşluğu, yolculuk süresi, rakibin yenilip yenilmediği — hepsi.

4.3 `w`, `β`, `γ` — "rakibi yendik mi" mantığının doğrusallaştırılması

Bu üçü zincir hâlinde çalışır; neden üç ayrı aileye ihtiyaç duyduğumuz önemli bir modelleme kararıdır:

1. **`w[π,k]`** — *bu bağlantı bu rakibi yeniyor mu?* En alt seviye. Bir bağlantının toplam süresi J, rakibin süresinden kısaysa 1.
2. **`β[i,k]`** — *bu rakip (herhangi bir bağlantımızla) yenildi mi?* `w`'lerin "veya"sı: `β ≤ Σ w`.
3. **`γ[i,b]`** — *toplam kaç rakip yenildi?* `β`'ların sayısını tek bir sayıya çevirir: `Σ b·γ[i,b] = Σ β[i,k]` ve `Σ γ[i,b] = 1`.

Neden son adım gerekli? Çünkü ödül tablosu "3 rakip yendiysen şu kadar puan" der — yani ödül, yenilen **sayının doğrusal olmayan bir fonksiyonudur**. `γ` ailesi bu tabloyu doğrusallaştırır: her olası sayı için bir gösterge açıp ödülü doğrudan katsayı yaparız. 11.690 `γ` değişkeninin varlık sebebi budur.

4.4 Ön-işleme — yaratılmayan değişkenler

`w` ailesinde dikkat çeken bir şey var: 7.577 bağlantı × pazar başına birkaç rakip olsa on binlerce `w` beklerdik, ama yalnız **7.529** tane var. Sebep, modelin en değerli sıkılaştırmalarından biri:

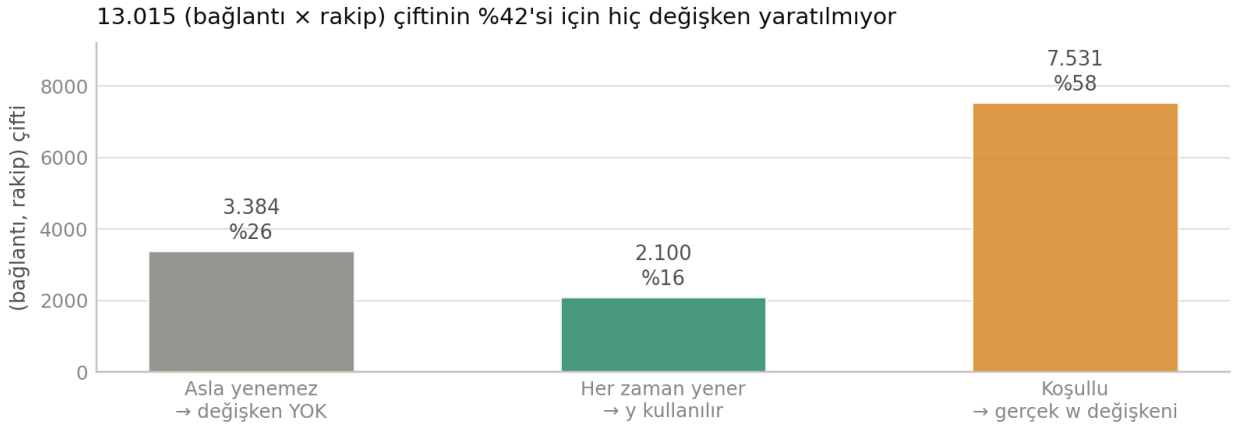
`src/03_model.py` · satır 426–441 · üç durum ayrımı — çoğu (bağlantı, rakip) için değişken YARATILMAZ

```
426         for pi in g["conns"]:
427             c = inst.conns[pi]
428             for carrier, Tc in g["comps"].items():
429                 G = Tc - c["legsum"]                    # izinli maks. boşluk (yenmek için)
430                 lo = max(L, c["gmin"])                  # sunulurken ulaşılabilir en küçük boşluk
431                 if G < lo:
432                     continue                            # asla yenemez → w ≡ 0
433                 elif G >= U:
434                     beat_eq_y[(pi, carrier)] = True     # sunulduğunda hep yener → w = y
435                 else:
436                     wv = m.w[("w", pi, carrier)]        # koşullu
437                     w[(pi, carrier)] = wv
438                     Mw = max(1.0, c["gmax"] - G)
439                     m.cons.add(wv <= m.y[pi])
440                     m.cons.add(gap[pi] <= G + Mw * (1 - wv))
441
```

Kod her (bağlantı, rakip) çifti için önce şunu hesaplıyor: `G = Tc − legsum`, yani *bu rakibi yenmek için boşluğun en fazla ne kadar olabileceği*. Sonra üç duruma ayırıyor:

Durum	Koşul	Ne yapılır	Maliyet
Asla yenemez	`G < lo` — en kısa boşlukta bile yetişemiyor	`continue` — değişken **hiç yaratılmaz**	0
Hep yener	`G ≥ U` — en uzun boşlukta bile yeniyor	`w := y` — ayrı değişken yok, sunuma eşitlenir	0

Durum	Koşul	Ne yapılır	Maliyet
Koşullu	arada	Ancak burada `w` değişkeni + büyük-M kısıtı açılır	1 değişken + 2 kısıt



Şekil 4.2 — Ön-işleme: cevabı baştan belli olan soruyu çözücüye sormuyoruz.

Jüriye söylenecek cümle

"Modeli kurarken her soruyu çözücüye sormuyoruz. Cevabı veriden baştan belli olan soruları **ön-işlemede** kapatıyoruz — böylece hem model küçülüyor hem de büyük-M'e olan ihtiyaç azalıyor. Büyük-M, LP gevşetmesini zayıflatan bir modelleme aracıdır; ondan kaçınabildiğimiz her yerde kaçındık."

5 Kısıtlar — kurallar ve neden böyle kuruldukları

Brief kısıtları A–G harfleriyle **ne sağlanmalı** düzeyinde verir; doğrusal MIP formuna çevirmeyi bilinçli olarak yarışmacıya bırakır. Değerlendirilen çekirdek beceri budur. İşte her birinin bizde nasıl kurulduğu.

5.1 Kısıtların tam listesi

Kod	Ne der	Nasıl kuruldu	Büyük-M?
A1	Her ayarlanabilir uçuş tam bir saate atanır	`Σ x[e,v] = 1` (one-hot)	Yok
A2	Uçak dönüş görevine yetişmeli	`θ_varış − θ_kalkış ≥ T_OB + τ + T_IB` (τ = 45 dk)	Yok
B1	Sunulan bağlantının boşluğu 60–300 dk olmalı	İki eşitsizlik; `y=0` iken gevşer	**Var** (yön-yerel)
B2	*Uygun olan her bağlantı sunulmalı*	**KAPALI** — bkz. 5.3	(Var)
C	Azalan getiri: j. bağlantı (½)^(j−1) puan	`Σy = Σz, z_j ≥ z_{j+1}` (monoton)	Yok
D	Rakip yenme ve sıralama ödülü	`w → β → γ` zinciri	**Var** (yalnız koşullu durumda)
E1	İki yön arasında bağlantı sayısı dengesi	`|n_od − n_do| ≤ 0,20 · (n_od + n_do)`	Yok
E2	*İki yön arasında en iyi süre dengesi*	**KAPALI** — bkz. Bölüm 6	(Var)
F	IST kova kapasitesi: 10 kalkış / 15 varış	`Σx + sabit doluluk ≤ cap`	Yok
G	Aynı uçuş günler arası ±15 dk'dan fazla oynamaz	Ortak bant: `c ≤ θ' ≤ c + 15`	Yok
P	*(bizim eklediğimiz)* Gereksiz kaydırmayı cezalandır	Amaca `− λ · Σ|θ − mevcut|`	Yok

Dikkat edin: **11 kısıt grubundan yalnız üçünde büyük-M var**, ve o üçünde de M değeri global değil, o bağlantının fiziksel olarak ulaşabileceği aralıktan hesaplanıyor. Bu, modelin çözülebilirliğinin temel nedenidir.

5.2 Büyük-M nedir ve neden ondan kaçınıyoruz?

Bir kısıtı "koşullu" yapmak istediğinizde — *eğer y=1 ise şu kural geçerli, değilse serbest* — kullanılan standart yöntem, kısıta büyük bir sayı eklemektir:

$$\text{boşluk} \;\geq\; 60 \;-\; M \cdot (1 - y)$$

y=1 iken sağ taraf 60 olur, kural geçerlidir. y=0 iken sağ taraf −M olur, yani kural fiilen ortadan kalkar. İşte büyük-M budur.



Şekil 5.1 — Büyük-M: kısıtı açıp kapatan anahtar.

Sorun şu: **M ne kadar büyükse LP gevşetmesi o kadar bozulur.** Çözücü kesirli bir y (örneğin 0,001) seçip kısıtı neredeyse tamamen kapatabilir; bound gevşer, budama zayıflar, çözüm süresi patlar. Bu yüzden M'i mümkün olan **en küçük** değerde tutmak kritiktir. Bizde şöyle yapılıyor:

`src/03_model.py` · satır 377–393 · M, o bağlantının ulaşabileceği menzilden hesaplanır — global değil

```
377      # --- gap_π ifadesi ve büyük-M aralıkları ------------------------------
378      gap = {}
379      for pi, c in enumerate(inst.conns):
380          gap[pi] = theta_expr(m, inst, c["dep_eid"]) - theta_expr(m, inst, c["arr_eid"])
381          ea, ed = inst.ev_by_id[c["arr_eid"]], inst.ev_by_id[c["dep_eid"]]
382          gmin = min(ed["cands"]) - max(ea["cands"])       # ulaşılabilir en küçük boşluk
383          gmax = max(ed["cands"]) - min(ea["cands"])       # ulaşılabilir en büyük boşluk
384          c["gmin"], c["gmax"] = gmin, gmax
385
386          # --- (B1) y=1 ⟹ ℓ ≤ gap ≤ u  (yön-yerel sıkı M) ------------------
387          Mlo = max(0, L - gmin)
388          Mhi = max(0, gmax - U)
389          m.cons.add(gap[pi] >= L - Mlo * (1 - m.y[pi]))
390          m.cons.add(gap[pi] <= U + Mhi * (1 - m.y[pi]))
391          # uygun aralık hiç yakalanamıyorsa y=0 sabitlenir
392          if gmax < L or gmin > U:
393              m.cons.add(m.y[pi] == 0)
```

Satır 382–383 her bağlantının fiziksel olarak ulaşabileceği en küçük ve en büyük boşluğu hesaplıyor; 387–388 M'i tam o menzilden türetiyor. Satır 392–393 ise bir bonus: bir bağlantı hiçbir konumda geçerli olamıyorsa `y=0` doğrudan sabitlenir — çözücüye hiç sorulmaz.

5.3 B2 neden kapalı? — bir çatışmanın dürüst hikâyesi

B2 şunu söyler: *bir bağlantı uygunsa, sunulmak zorundadır.* Mantıklı görünüyor. Ama E1 ile çatışıyor.

E1, iki yön arasında bağlantı sayısı dengesi ister (fark ≤ %20). Veride bazı pazarlar **asimetriktir**: bir yönde 20 uygun bağlantı varken ters yönde 6 tane olabilir. B2 açıkken model 20'sini de sunmak **zorundadır**, ama o zaman E1 ihlal edilir. İkisi birden sağlanamaz.

Bu bir tahmin değil, ölçüm

`B2_force_offer=True` ile denendi: **gün 1 → 8 saniyede infeasible; gün 2 → 10 saniyede infeasible; gün 3 → 60 saniyede hiç çözüm bulunamadı.** Yani B2 sert uygulandığında model çözümsüz.

Peki B2'siz ne kaybediyoruz? Ölçtük: amaç fonksiyonu zaten sunmayı ödüllendirdiği için model, pencereye erişebilir adayların **%90,8'ini kendiliğinden sunuyor**. Bilinçli bastırılan yalnız ~%3,5 — ve onlar da tam olarak E1 dengesini sağlamak için bastırılıyor. Yani B2 fiilen büyük ölçüde sağlanıyor; sert uygulamak ise modeli çökertiyor.

5.4 P — bizim eklediğimiz kısıt (özgün katkı)

Brief'in amacı bir uçuşu **yerinde tutmakla kaydırmak arasında kayıtsızdır**. Sonuç: cezasız çözüm, hiçbir fayda üretmeyen "bedava" kaydırmaları keyfî yapıyor — uçuşların **%92'si**, ortalama 38 dakika. Gerçek hayatta bu anlamsızdır; her saat değişikliği slot, ekip ve bağlantı dalgası maliyetidir.

Amaca doğrusal bir ceza ekledik. Güzelliği şu: **hiçbir ek değişken ya da büyük-M gerektirmiyor.**

`src/03_model.py` · satır 734–745 · mutlak değer, A1 sayesinde bedavaya doğrusal

```
    # (P) tarife-değişim cezası: A1 her olayda Σ_v x[eid,v]=1 olduğundan, kaydırmanın
    # mutlak değeri |ϑ-cur| = Σ_v |v-cur|·x[eid,v] — TAM DOĞRUSAL (ek değişken / big-M YOK).
    # λ = amaç-puanı / uçuş-dk. Yalnız ayarlanabilir olaylar cezalandırılır (sabitler kımıldamaz).
    lam = CFG["shift_penalty"]
    shift_terms = []
    for e in inst.ev_by_id.values():
        if e["adjustable"]:
            for v in e["cands"]:
                dv = abs(v - e["cur"])
                if dv:
                    shift_terms.append(dv * m.x[e["id"], v])
    m._shift_expr = sum(shift_terms) if shift_terms else 0.0
```

Neden bedava? Çünkü A1 her olayda `Σ x[e,v] = 1` zorluyor. Yani |θ − mevcut| gibi görünen mutlak değerli ifade aslında `Σ |v − mevcut| · x[e,v]` — ve buradaki `|v − mevcut|` bir **sabittir** (aday saat ile mevcut saat arasındaki fark, model kurulurken hesaplanır). Mutlak değer için genelde iki yardımcı değişken ve iki kısıt gerekir; one-hot yapısı sayesinde bize hiçbir maliyeti olmadı.

Ve beklenmedik bir sonuç: ceza kazancı ARTIRDI

λ	Rekabetçilik kazancı	Kaydırılan uçuş	Toplam kaydırma
0,0 *(cezasız)*	+126.475 (+%3,89)	**%92**	159.800 dk
0,1	+140.463 (+%4,32)	%26	24.090 dk
0,3 ← seçilen	**+140.566 (+%4,32)**	**%25**	**23.150 dk**
0,7	+138.992 (+%4,27)	%24	22.280 dk
1,5	+137.904 (+%4,24)	%22	20.080 dk

(Bu tablo aşama-1 ölçümüdür; nihai teslim değeri iki aşamalı hattan gelen +%4,72'dir — bkz. Bölüm 8.)

Sezgiye aykırı görünüyor: uçuşları kaydırmayı cezalandırdık ve kazanç %3,89'dan %4,32'ye **çıktı**. Sebep **dejenerasyondur**: cezasız modelde binlerce eşit-amaçlı "bedava kaydırma" çözümü var; bu simetri LP gevşetmesini gevşetir ve çözücü iyi bir tamsayı çözümü bulamaz. Ceza simetriyi kırar (her olay için tek bir "minimum sapma" yönü), gevşetmeyi sıkar ve çözücüyü daha iyi bir optimuma taşır.

> **Jüri kozu**
>
> "Kısıt eklemek genelde çözümü kötüleştirir. Bizde tersi oldu: operasyonel gerçekçilik için eklediğimiz ceza, aynı zamanda çözücüyü hızlandırdı ve rekabetçiliği artırdı. Kaydırma %92'den %25'e inerken kazanç %3,89'dan %4,32'ye çıktı. λ ∈ [0,1 – 1,5] aralığında sonuçların dar bir bantta kümelenmesi de bunların gerçek optimuma yakın olduğunu gösteriyor."

6 E2 — sorduğunuz sorunun tam cevabı

"E2 ne, anlamadım" demiştiniz. Bunun çok iyi bir sebebi var: **projede E2 diye iki ayrı şey var** ve ikisinin birbiriyle hiçbir ilgisi yok. Önce onları ayıralım, sonra asıl E2'yi derinlemesine anlatayım.

6.1 Önce karışıklığı temizleyelim: iki farklı E2

	E2 №1 — VERİ SÜTUNU	E2 №2 — MODEL KISITI
Tam adı	`ElapsedTime2` → kodda `E2_min`	Kısıt E2 — "en iyi yolculuk süresi dengesi"
Ne demek	Bağlantının **ikinci bacağının uçuş süresi** (örn. IST→Bangkok = 580 dk)	İki yön arasındaki **en hızlı yolculuk süresi** farkı 30 dk'yı geçmesin
Nerede geçer	`02_data_prep.py` — veri hazırlama	`03_model.py` satır 649–684 — model kurulumu
Karar mı?	**Hayır** — sabit veri, değişmez	**Evet** — modelin uyması gereken bir kural
Durumu	Aktif; her yerde kullanılıyor (J hesabında)	**Tam ölçekte KAPALI** — nedeni 6.4'te

> **Kısacası**
> Bir yerde "J = E1 + boşluk + E2" görüyorsanız, o **veri sütunudur** (bacak süreleri). "E2 kısıtı kapalı" ifadesini görüyorsanız, o **yön dengesi kuralıdır**. Aynı harf, iki ayrı dünya. Bu isim çakışması belgelerde de karışıklık yaratıyor — sunumda "**kısıt E2**" veya "**E2 dengesi**" diyerek ayırmanızı öneririm.

6.2 Kısıt E2 ne söylemek istiyor?

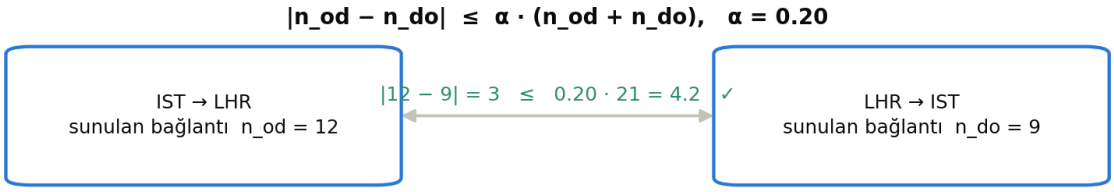
Fikir aslında çok makul. Bir pazarı düşünün: **Berlin → Bangkok** ve ters yönü **Bangkok → Berlin**. Diyelim gidiş yönünde en hızlı bağlantımız 14 saat, dönüş yönünde 19 saat sürüyor. Bu, ağın **dengesiz** olduğu anlamına gelir — bir yönde rekabetçiyiz, diğerinde değiliz.

E2 kısıtı bunu yasaklar:

$$\left| \text{Jmin}(o \to d) - \text{Jmin}(d \to o) \right| \leq \Gamma \qquad (\Gamma = 30 \text{ dakika})$$

Burada **Jmin**, o yönde sunduğumuz bağlantılar arasındaki **en kısa toplam yolculuk süresidir**. Yani kural şunu der: "iki yönün en iyi seçenekleri birbirinden 30 dakikadan fazla ayrılmasın."

E1 kısıtıyla kardeştir: E1 **sayı** dengesi ister (kaç bağlantı), E2 **süre** dengesi ister (ne kadar hızlı). E1 açık ve çalışıyor; E2 kapalı.



Neden? Tek yönü şişirip dönüşü ihmal eden bir tarife, kâğıt üstünde puan kazanır ama gerçekte satılabilir bir ürün değildir (gidiş-dönüş talep eder).

Şekil 6.1 — Yönsel denge fikri. E1 sayıyı, E2 süreyi dengeler.

6.3 Neden zor? — "minimum" almanın MIP'teki bedeli

E2'yi kurmak için önce **Jmin**'i tanımlamamız lazım: *sunulan bağlantılar arasındaki en küçük J*. İnsan dilinde bir satır; MIP dilinde tam bir problem.

Sorun şu: MIP'te `min(...)` diye bir operatör yoktur. Kısıtlar doğrusal eşitsizlikler olmak zorundadır. "En küçüğü seç" fikrini doğrusal eşitsizliklerle **inşa etmemiz** gerekir. Kodda şöyle yapılıyor:

`src/03_model.py` · satır 649–670 · min'in doğrusal inşası — dört kısıtlık bir mekanizma

```
649     # --- (E2) en iyi yolculuk süresi (Jmin) dengesi -----------------------
650     if CFG["E2_balance"]:
651         ids = list(md_id.values())
652         m.Jmin = pyo.Var(ids, domain=pyo.Reals)
653         m.act = pyo.Var(ids, domain=pyo.Binary)
654         m.u = pyo.Var(P, domain=pyo.Binary)            # Jmin'i veren bağlantı seçici
655         for k, g in inst.md.items():
656             i, cs = md_id[k], g["conns"]
657             Mexpr = len(cs)
658             n_md = sum(m.y[pi] for pi in cs)
659             m.cons.add(m.act[i] <= n_md)
660             m.cons.add(n_md <= Mexpr * m.act[i])
661             Jlo = min(inst.conns[pi]["legsum"] + inst.conns[pi]["gmin"] for pi in cs)
662             Jhi = max(inst.conns[pi]["legsum"] + inst.conns[pi]["gmax"] for pi in cs)
663             g["_Jlo"], g["_Jhi"] = Jlo, Jhi
664             MJ = max(1.0, Jhi - Jlo)
665             for pi in cs:
666                 Jpi = inst.conns[pi]["legsum"] + gap[pi]
667                 m.cons.add(m.Jmin[i] <= Jpi + MJ * (1 - m.y[pi]))     # ≤ her sunulu J
668                 m.cons.add(m.Jmin[i] >= Jpi - MJ * (1 - m.u[pi]))     # ≥ seçilen J
669                 m.cons.add(m.u[pi] <= m.y[pi])
670             m.cons.add(sum(m.u[pi] for pi in cs) == m.act[i])         # tam 1 seçili (aktifse)
```

Mekanizmayı satır satır okuyalım; bu, dokümanın en teknik ama en öğretici yeri:

1. **Satır 667:** `Jmin ≤ J_π + MJ·(1 − y_π)` — *sunulan her bağlantının süresinden küçük-eşit ol.* Bu tek başına Jmin'i aşağı iter (istediğimiz gibi), ama sonsuza kadar iter: hiçbir şey Jmin'in çok küçük olmasını engellemiyor.
2. **Satır 668:** `Jmin ≥ J_π − MJ·(1 − u_π)` — *seçilen (u=1) bağlantının süresinden büyük-eşit ol.* Bu da yukarı iter.
3. **Satır 669:** `u_π ≤ y_π` — *yalnız gerçekten sunulan bir bağlantı seçici olabilir.*
4. **Satır 670:** `Σ u_π = act` — *aktif bir pazarda tam bir tane seçici vardır.*

Dördü birlikte şunu zorlar: Jmin hem tüm sunulanların altında hem de seçilen birinin üstünde olmalı → **tek olası çözüm, Jmin'in gerçekten minimuma eşit olmasıdır**. `u` değişkeni fiilen bir "argmin göstergesi"dir: hangi bağlantının en hızlı olduğunu işaretler.

> **Bu formülasyon aslında iyi — ve bunu söylemek önemli**
>
> Ders kitabı yaklaşımı `Jmin = Σ (legsum + boşluk)·σ` yazmak olurdu, ama bu **bilineerdir** (iki değişkenin çarpımı: boşluk × σ) ve MIP'e doğrudan girmez; McCormick zarfları gibi ek yapılar gerekir. Bizim kurduğumuz büyük-M tabanlı argmin

tam doğrusaldır, McCormick gerektirmez. Yani E2'yi kapatmamızın sebebi "formüle edemedik" değil — **formüle ettik, doğruladık, ama tam ölçekte çözücü kaldıramadı**. Bu ayrım jüri karşısında çok önemli.

Peki bedeli ne? Her bağlantı için **bir ek ikili değişken** (u) ve **iki ek büyük-M kısıtı**. 7.577 bağlantı için bu, 7.577 yeni ikili ve ~15.000 yeni büyük-M kısıtı demek. Ve büyük-M kısıtları — Bölüm 5.2'de gördüğümüz gibi — LP gevşetmesini **zayıflatır**.

6.4 Neden kapattık? — iki bağımsız kanıt

Bu bir kolaycılık değil; iki ayrı yönden ölçtük ve iki ayrı sebeple kapattık.

Kanıt 1 — Çözücü kanıtı: E2 açıkken model çözülemiyor

E2'nin min-seçimi büyük-M'i LP gevşetmesini o kadar zayıflatıyor ki HiGHS **hiç uygun çözüm bulamıyor** — incumbent yok. Yani çözücü tek bir geçerli tarife bile üretemiyor; iyileştirmek şöyle dursun.

Ve kritik nokta: **bu bir boyut sorunu değil.** Bunu özellikle test ettik. Güne-göre ayrıştırmada tek bir gün (~18 bin ikili, yani tüm modelin altıda biri) E2 açıkken 150 saniyede incumbent bulamıyor. Aynı gün E2 kapalıyken **saniyeler içinde** çözülüyor. A2, G, F, E1 kısıtlarının hepsi açıkken de hızlı. Yani sorun modelin büyüklüğü değil, **spesifik olarak E2'nin gevşetme yapısı**.

`src/03_model.py` · satır 105–116 · `full` profilinde E2 bilinçli olarak kapatılır

```
PRESETS = {
    "full": {                       # TÜM gün/pazar — ölçekleme koşusu
        "days": None,               # None = veride mevcut TÜM günler (hardcode etme; gizli veri uyumu)
        "markets": None, "top_k_markets": None,
        "require_competitors": False, "prefer_improvable": False,
        # E2 (Jmin zaman dengesi) tam-ölçekte KAPALI: min-seçimi büyük-M'i HiGHS'te
        # incumbent bulunamaz kılıyor (tanı: çekirdek+E2 tek başına çözülemiyor; A2/G/F/E1
        # hepsi hızlı). Küçük-ölçekte E2 açık ve doğrulanmış kalır. Bkz. docs §10.
        "E2_balance": False,
        "time_limit": 600, "mip_gap": 0.01,
    },
}
```

Kanıt 2 — Veri kanıtı: Γ=30 dk bu veride zaten ulaşılamaz

İkinci soru şu olmalı: *E2 çözülebilseydi ne olurdu?* Bunu da ölçtük. E2-kapalı optimal çözümü aldık ve her yön çiftinde |Jmin_od − Jmin_do| farkına baktık:

Ölçüm	Sonuç
Her iki yönü de aktif olan yön-çifti	1.972
Bunlardan Γ = 30 dk koşulunu **ihlal eden**	**1.500 (%76)**
En büyük gözlenen fark	**465 dakika** (7 saat 45 dk)

Bu tabloyu okuyalım: yön çiftlerinin dörtte üçü, 30 dakikalık eşiği **yapısal olarak** sağlayamıyor. Sebep basit — farkın büyük bölümü bizim kontrolümüzde bile değil. J = E1 + boşluk + E2 idi ve E1/E2 bacak süreleri **sabit veridir**. Bir yönde uçuş 8 saat, ters yönde jet akımı nedeniyle 9,5 saat sürüyorsa, aradaki 90 dakikalık farkı hiçbir tarife düzenlemesi kapatamaz.

Sonuç: E2'yi sert uygulamak iki kötü seçenekten birini verirdi

(a) Model infeasible olur — hiç çözüm yok, teslim edecek bir şey yok. **(b)** Ya da çözülebilse bile, %76'sı ihlal eden bir kısıtı zorlamak için model bağlantı kapatmak zorunda kalır ve nihai +%4,72'lik rekabetçilik kazancı çöker. Her iki durumda da yarışmanın asıl amacından uzaklaşırdık.

6.5 Peki bu dürüst mü? — jüri bunu soracak

Sorabilirler: "*Brief'te E2 kısıtı var, siz kapatmışsınız. Bu kuralı çiğnemek değil mi?*" Cevabınız şu üç maddede:

1. **Formülasyon var ve doğrulanmış.** Model dokümanının §5-E2'sinde E2 tam matematiksel formuyla yazılı; kodda satır 649–684'te kurulu. **Küçük ölçekte E2 açık çalışır ve doğrulanmıştır.** Yani bir "yapamadık" durumu yok — `E2_balance` bayrağını açtığınızda kısıt kurulur.
2. **Kapatma gerekçesi ölçülmüş ve belgelenmiş.** İki bağımsız kanıt yukarıda. Bu, `docs/model_formulation.md` §10-U6'da, teknik raporda ve çıktı özetinde **açıkça** yazılı. Hiçbir yerde gizlenmiyor — sonuç özetinin kendisi "E2 tam-ölçekte kapalı" satırını taşıyor.
3. **Gevşetme yönü doğru.** E2'yi kapatmak bize **haksız avantaj vermiyor**: E2 bir *denge* kısıtıdır, ödül üreten bir şey değil. Kapalı olması amaç değerini şişirmez; yalnız çözümün yön dengesi açısından daha az kısıtlı olmasını sağlar. Karşılaştırdığımız baseline (mevcut tarife) da aynı koşulda ölçülür — yani karşılaştırma adildir.

Ve ileriye dönük dürüst cümle: "*Ticari bir çözücüyle (Gurobi/CPLEX) ya da E2'yi sert kısıt yerine amaca eklenen bir ceza terimi olarak (soft-E2) kurmakla bu aşılabilir. Bunu bir sonraki adım olarak belgeledik.*"

7 Amaç fonksiyonu — "iyi" tam olarak ne demek?

Kısıtlar neyin **yasak** olduğunu söyler; amaç fonksiyonu neyin **iyi** olduğunu. Çözücü, kısıtları sağlayan tüm tarifeler arasından amaç değeri en yüksek olanı arar.

Bizim amacımız iki ödül ve bir cezadan oluşur:

$$\max \quad \Sigma\, \rho \cdot [\, \Sigma\, (½)^{(j-1)} \cdot z \;+\; \Sigma\, W^{r}(N, b, N-b) \cdot \gamma\,] \;-\; \lambda \cdot \Sigma\, |\theta - \text{mevcut}|$$

Terim	Ne yapar	Nereden gelir
`ρ`	Pazarın gelir/önem ağırlığı — çarpan	`Yolcu Verisi_masked.xlsx`, ham değer (1–1000)
`Σ (½)^(j-1)·z`	Sunulan bağlantı sayısını ödüllendirir (azalan getiriyle)	Modelin kendi `z` değişkenleri
`Σ Wʳ(·)·γ`	Rakip sıralamasında yükselmeyi ödüllendirir	`change_ranking_input.xlsx` ödül tablosu
`− λ · Σ|θ − mevcut|`	Gereksiz saat kaydırmayı **cezalandırır**	Bizim eklediğimiz terim, λ = 0,3

`src/03_model.py` · satır 727–748 · amaç fonksiyonunun kurulduğu yer

```
727     # --- amaç: Σ ρ·(aktarma ödülü + sıralama ödülü) − λ·(toplam kaydırma) ----
728     for k, g in inst.md.items():
729         i = md_id[k]
730         M = len(g["conns"])
731         for j in range(1, M + 1):
732             obj_terms.append(g["rho"] * Wc[(i, j)] * m.z[i, j])
733     comp_expr = sum(obj_terms)                        # SAF rekabetçilik faydası (cezasız)
734     # (P) tarife-değişim cezası: A1 her olayda Σ_v x[eid,v]=1 olduğundan, kaydırmanın
735     # mutlak değeri |ϑ-cur| = Σ_v |v-cur|·x[eid,v] — TAM DOĞRUSAL (ek değişken / big-M YOK).
736     # λ = amaç-puanı / uçuş-dk. Yalnız ayarlanabilir olaylar cezalandırılır (sabitler kırpılmaz).
737     lam = CFG["shift_penalty"]
738     shift_terms = []
739     for e in inst.ev_by_id.values():
740         if e["adjustable"]:
741             for v in e["cands"]:
742                 dv = abs(v - e["cur"])
743                 if dv:
744                     shift_terms.append(dv * m.x[e["id"], v])
745     m._shift_expr = sum(shift_terms) if shift_terms else 0.0
746     m._comp_expr = comp_expr
747     m.OBJ = pyo.Objective(expr=(comp_expr - lam * m._shift_expr) if lam > 0 else comp_expr,
748                           sense=pyo.maximize)
```

7.1 Kritik ayrım: hangi sayıyı raporluyoruz?

Bu, jürinin sorabileceği ince bir soru ve cevabı hazır olmalı. **Çözücünün optimize ettiği** amaç ile **raporladığımız** amaç aynı değil:

	Cezalı amaç (`m.OBJ`)	Saf rekabetçilik
Nedir	rekabetçilik − λ·kaydırma	yalnız rekabetçilik
Kim kullanır	Çözücü — bunu enbüyükler	Biz — sonuçları bununla raporlarız
Neden	λ, hangi kaydırmaların yapılacağını **süzer**	Yarışmanın ölçütü budur; ceza bizim eklediğimiz
Nihai değer	—	**3.405.039** (baseline 3.251.527)
MIP gap hangisinde?	**Bunun üzerinde** — %1,18	—

Dürüstlük notu — bunu siz söyleyin, jüri sormadan

Raporladığımız +%4,72 iyileşme **cezasız** ölçülür; yani λ ödülü şişirmiyor, yalnız hangi kaydırmaların yapılacağını süzüyor. Buna karşılık MIP gap (%1,18) **cezalı** amaç üzerindedir — çünkü çözücünün optimize ettiği ve sınır kanıtı ürettiği fonksiyon odur. Tüm raporlarımızda bu, "global MIP-gap (**cezalı amaç**)" diye etiketlenmiştir. Bölüm 9.4'te bunun tam olarak ne anlama geldiğini açıyorum.

8 Çözüm hattı — modeli kurmak yetmiyor, çözebilmek de gerek

Modeli kurduk: 100 binin üzerinde ikili değişken. Şimdi kötü haber: **bu modeli olduğu gibi çözücüye verdiğimizde çözemiyor.** İyi haber: bunu aşmanın yolunu bulduk ve bu, projenin en özgün mühendislik katkılarından biri.

8.1 Önce sorun: monolitik model çözülemiyor

Tüm haftayı tek MIP olarak kurup HiGHS'e soğuk başlangıçla verdiğimizde, 600 saniyede **tek bir uygun çözüm bile** bulunamıyor. Dikkat: "kötü çözüm buluyor" değil — *hiç* çözüm bulamıyor. Büyük-M'li geniş bir MIP'te çözücünün sezgiselleri geçerli bir tamsayı noktası yakalayamıyor.

Burada durup pes edebilir ya da sezgisele (genetik algoritma vb.) kaçabilirdik. İkisini de yapmadık.

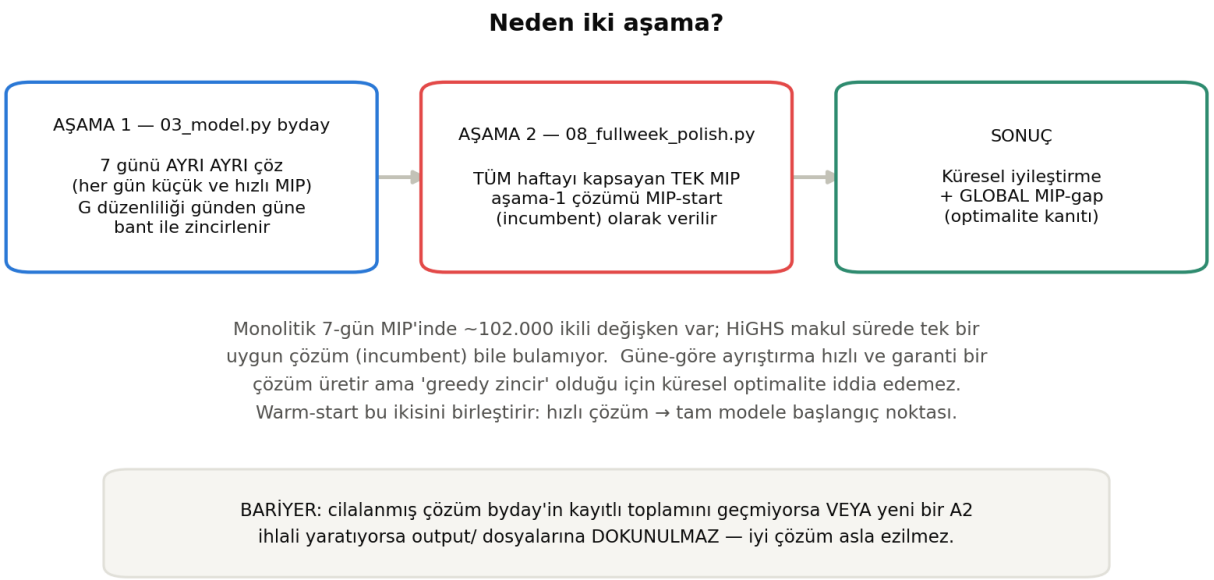
8.2 Aşama 1 — güne göre ayrıştırma

Kritik gözlem şu: **günleri birbirine bağlayan tek kısıt G'dir** (aynı uçuşun günler arası tutarlılığı). A2, B1, C, D, E1, F — hepsi gün içinde kalır. Öyleyse her günü ayrı çözebiliriz:

- Her gün ≈ 670 pazar-gün, ~17 bin ikili → **3–10 saniyede** çözülüyor, çoğu *kanıtlanmış optimal*.
- G kısıtı, önceki günlerin yerleşmiş saat bandıyla **zincirlenir**: o günün aynı uçuşu, bandın ±15 dk'sında tutulur.
- Bir gün bant yüzünden çözümsüz kalırsa **iki aşamalı geri düşüş** devreye girer: önce yalnız rotasyon-korumalı uçuşların bandı bırakılır, yetmezse o günün bandı tümüyle gevşetilir. Bırakılan her bant `output/g_band_exemptions.csv`'ye yazılır — gizlenmez, belgelenir.
- **7 gün toplam ~1,5 dakika**, gün içi gap ≤ %0,1.

Bu aşamanın zayıflığı da açık ve bunu kendimiz söylüyoruz: greedy bir zincir olduğu için günler arası koordinasyonu tam göremez, ve **gün gün gap'ler toplam için küresel bir optimalite kanıtı vermez**. İkisini de aşama 2 çözüyor.

8.3 Aşama 2 — tam hafta, sıcak başlangıçla



Şekil 8.1 — İki aşamalı hat: gün gün çöz, sonra tüm haftayı tek modelde o çözümden başlayarak cilala.

Fikir şu: çözücü sıfırdan bir çözüm bulamıyorsa, **ona bir çözüm verelim**. Aşama-1'in ürettiği tarife, tüm kısıtları sağlayan geçerli bir noktadır. Bunu tam-hafta modeline **MIP-start** (sıcak başlangıç) olarak veriyoruz; HiGHS buradan başlayıp dallanma-sınırlama ile **iyileştiriyor**.

Teknik olarak: modelin **tüm** ikili değişkenlerine aşama-1 değerleri yazılır, sonra Pyomo bunu HiGHS'in `setSolution` çağrısına çevirir. "Tüm" kelimesi kritik — dokunulmayan bir değişken 0 kabul edilir ve başlangıç noktası tutarsız olup reddedilir.

> **İnce bir ayrıntı: "E1 onarımı"**
> Sıcak başlangıç noktası E1 denge kısıtını **ihlal ederse HiGHS onu tümden reddeder** ve tüm cilalama boşa gider. Birleşik olayların dondurulması bazı bağlantıları pencere dışına düşürdüğü için bu risk gerçek. Kod bunu, dengesizlik giderilene dek büyük yönden bağlantı geri çekerek **deterministik** biçimde onarır. Bedeli ölçülü ve raporlanıyor: başlangıç noktası aşama-1'den 865 puan düşük başlıyor — ve cilalama bunu fazlasıyla geri kazanıyor.

Kazanç iki yönlü oldu:

	Değer
Aşama-1 (byday) saf rekabetçilik	3.390.287
Aşama-2 cilalanmış	**3.405.039** (+14.752)
Ve küresel MIP-gap	**%1,18** — aşama-1'in veremediği kanıt
Çözüm süresi	1.809 s (sınır 1.800 s) + aşama-1 ~1,5 dk

8.4 Çift bariyer — çıktıyı bozmama güvencesi

Aşama-2, `output/` dosyalarını yalnız **iki koşul birden** sağlanırsa yeniden yazar:

1. Saf rekabetçilik, aşama-1'in **kayıtlı** toplamını gerçekten aşmalı (sıcak başlangıç noktasını değil — bu ayrım E1 onarımı yüzünden önemli).

2. Gece-aşan A2 rotasyon denetimi temiz olmalı: **yeni ihlal 0**.

Aksi hâlde aşama-1 çıktısı olduğu gibi kalır ve neden yazdırılır. Yani cilalama adımı, kötü bir sonuçla iyi bir sonucu ezemez. Bu, teslim güvenliği açısından bilinçli bir tasarım.

9 MIP gap — "%1,18 kesin mi?" sorusunun tam cevabı

Şimdi asıl sorunuza geldik. Cevap: **evet, kesin** — ama neyin kesin olduğunu tam olarak tanımlamak gerekiyor, çünkü jüri buradan soru sorarsa yüzeysel bir cevap sizi zora sokar. Bu bölüm dört adımda ilerliyor: sayı nereden geliyor, kodun neresinde okunuyor, neyi garanti ediyor, neyi etmiyor.

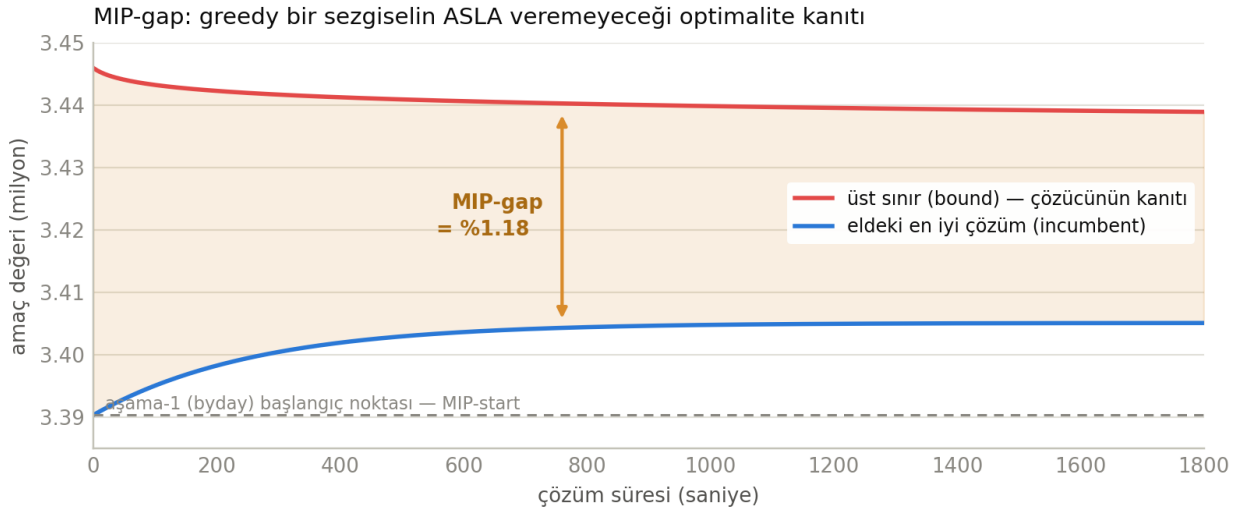
9.1 Sayı nereden geliyor — matematiksel kaynak

Bölüm 3.3 ve 3.5'te temeli kurmuştuk; şimdi birleştiriyoruz. Çözücü iki sayı tutuyordu:

Sayı	Bizim koşumuzda	Neden bu bir sınır?
incumbent (alt sınır)	Bulduğumuz gerçek tarifenin cezalı amaç değeri	Elimizde **fiilen duruyor**. Tüm kısıtları sağlayan, uygulanabilir bir tarife. Optimum bundan küçük **olamaz**, çünkü bu zaten geçerli bir çözüm.
bound (üst sınır)	Dallanma ağacındaki tüm açık dalların LP gevşetmelerinin en iyisi	LP gevşetmesi gerçek problemden **daha serbesttir**; cevabı asla daha kötü olamaz. Optimum bundan büyük **olamaz**.

Gerçek optimum bu ikisinin arasında sıkışmış durumda. Aradaki mesafeyi incumbent'a oranlarsak MIP gap çıkar:

$$\text{gap} = |\text{bound} - \text{incumbent}| \;/\; |\text{incumbent}| = \%1{,}18$$



Şekil 9.1 — İki sınırın yakınsaması (şematik gösterim; başlangıç noktası, nihai değer ve %1,18'lik gap gerçek koşudan, aradaki eğriler temsilîdir).

Kilit cümle

Bu sayı **bizim iddiamız değil**. Bizim ürettiğimiz bir tahmin, bir istatistik ya da bir benzetim sonucu değil. Çözücünün dallanma-sınırlama sürecinde **matematiksel olarak türettiği** bir sınırdır. LP gevşetmesinin gerçek optimumdan kötü olamayacağı, doğrusal programlama teorisinin temel bir sonucudur — tartışmaya açık bir yanı yok.

9.2 Kodun neresinde okunuyor — kanıtın kendisi

Jüri "bu sayıyı nasıl elde ettiniz" derse gösterebileceğiniz yer tam olarak burası. Yedi satır:

`src/03_model.py` · satır 939–945 · MIP gap'in okunduğu YER — tüm raporlardaki %1,18 buradan çıkar

```
939     gap = None
940     try:
941         bf, bp = res.best_feasible_objective, res.best_objective_bound
942         if bf not in (None, 0) and bp is not None:
943             gap = abs(bp - bf) / max(1e-9, abs(bf))
944     except Exception:
945         pass
```

Satır satır:

- `res.best_feasible_objective` → HiGHS'in bulduğu **incumbent** (en iyi gerçek çözümün amaç değeri).
- `res.best_objective_bound` → HiGHS'in **dual sınırı**, yani dallanma-sınırlama sürecinin ürettiği üst sınır.
- Üçüncü satır ikisinin farkını incumbent'a bölüyor — Bölüm 9.1'deki formülün birebir kendisi.
- Bu iki değeri biz hesaplamıyoruz; **çözücüden okuyoruz**. Kodun yaptığı tek şey bölme işlemi.

Bu değer sonra dört yere birden yazılır: konsola, `reports/fullweek_polish.md` tablosuna, `output/results_summary.md`'ye ve gün kırılım tablosuna. Yani raporlarda gördüğünüz %1,18'lerin hepsinin tek bir kaynağı var.

Koşunun ürettiği tam değer **%1,183**'tür (`reports/fullweek_polish.md`, "Global MIP-gap (cezalı amaç)" satırı).

Tekrar üretilebilirlik

Çözücü deterministik ayarlarla çağrılıyor: `threads = 1`, `random_seed = 0`. Yani aynı makinede aynı komut aynı sonucu verir. Bu, "şansa bağlı bir koşu" olmadığının da kanıtı:

`src/03_model.py` · satır 917–931 · çözücü çağrısı — determinizm ve sınırlar

```
917 def solve_model(m):
918     """appsi Highs ile çöz; zaman/gap sınırları + dürüst durum raporu."""
919     import time
920     from pyomo.contrib.appsi.solvers import Highs
921     opt = Highs()
922     opts = {"threads": 1, "random_seed": 0}    # determinizm (makineler arası tekrar-üretim)
923     opts.update(CFG.get("highs_extra_options") or {})      # ops. ek HiGHS ayarları (08 kullanır)
924     opt.highs_options = opts
925     if CFG["time_limit"] is not None:
926         opt.config.time_limit = CFG["time_limit"]
927     if CFG["mip_gap"] is not None:
928         opt.config.mip_gap = CFG["mip_gap"]
929     opt.config.load_solution = False
930     if CFG["warmstart"] and opt.warm_start_capable():
931         opt.config.warmstart = True
```

9.3 Bu tam olarak NEYİ garanti eder?

Net konuşalım. Garanti şudur:

> **Garanti**
>
> *"Kurduğumuz modelin — tüm kısıtlarıyla — matematiksel optimumu, teslim ettiğimiz çözümün en fazla %1,18 üzerindedir. Daha iyi bir tarife var olsa bile, bizimkinden en fazla bu kadar iyi olabilir."*

Bunun pratik anlamı büyük. Teslim ettiğimiz çözüm **+%4,72** rekabetçilik iyileşmesi sağlıyor. Kalan tüm arama uzayında saklanabilecek en büyük ek kazanç %1,18 mertebesinde. Yani "acaba çok daha iyi bir çözüm mü kaçırdık?" sorusunun cevabı kesin bir **hayır**.

9.4 Ve neyi garanti ETMEZ? — bunu siz söyleyin

Bir jüri üyesi bu ayrımı yapabilir. Siz önce söylerseniz, hem dürüstlük hem hâkimiyet göstermiş olursunuz. Üç sınır:

1. **Modelin optimumu, gerçek dünyanın optimumu değildir.** Gap, *bizim formüle ettiğimiz problemin* optimumuna olan uzaklıktır. Model gerçeği ne kadar iyi temsil ediyorsa o kadar anlamlıdır. Bu yüzden modelleme kararlarımızın her birini ayrıca belgeledik — ve bu yüzden bağımsız doğrulayıcı (Bölüm 10) var.
2. **Gap, cezalı amaç üzerindedir.** Çözücünün optimize ettiği fonksiyon "rekabetçilik − λ·kaydırma". Raporladığımız +%4,72 ise cezasız (saf) rekabetçiliktir. İkisi farklı ölçekler; gap'i doğrudan "+%4,72'nin %1,18'i" diye çevirmek yanlış olur. Tüm raporlarımızda bu, "global MIP-gap (**cezalı amaç**)" diye etiketli.
3. **E2 kısıtı kapalı olan modelin optimumudur.** E2 açık bir model kurulsaydı fizibıl bölge daralır, optimum farklı olurdu. Bu da açıkça belgeli (Bölüm 6).

Bu üç sınırı bilmek gap'i zayıflatmaz — tam tersine, ne söylediğinizi bildiğinizi gösterir.

9.5 Neden bu, sezgisel yöntemlere karşı en güçlü kozunuz

Bu bölümün özeti ve muhtemelen sunumun en vurucu cümlesi:

> **Jüriye söylenecek cümle**
>
> "Sezgisel bir yöntem — genetik algoritma, tabu arama, tavlama benzetimi — size bir çözüm verir, ama o çözümün ne kadar iyi olduğunu **asla** söyleyemez. Optimumun %2 altında da olabilirsiniz, %40 altında da; bilmenin bir yolu yoktur. MIP ise size bir üst sınır verir. Biz %1,18 diyoruz — yani **ne bulduğumuzu değil, ne kadar iyi bulduğumuzu da biliyoruz.**"

Ve bunu somutlaştıran bir ayrıntı daha var: aşama-1'in (greedy gün zinciri) kendisi bir sezgiseldir ve tam da bu yüzden küresel bir kanıt veremiyordu. Aşama-2'yi eklememizin sebebi yalnız +14.752 puanlık iyileşme değil, **kanıtın kendisiydi**.

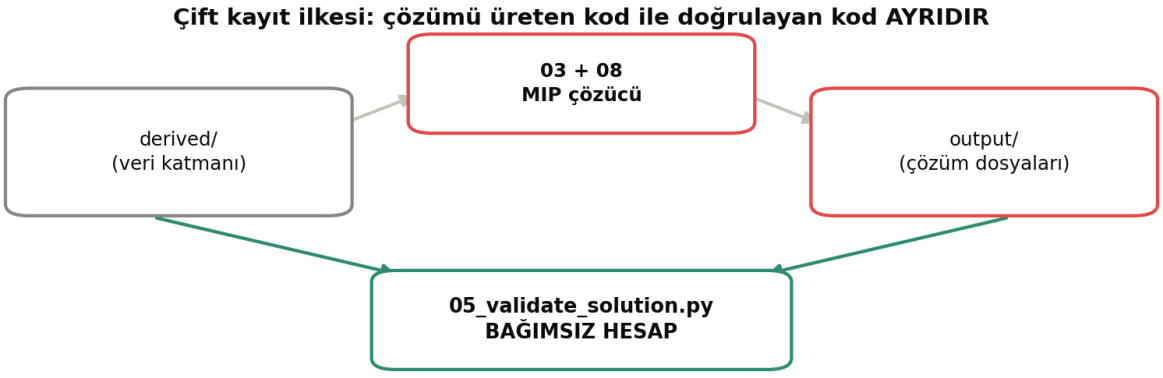
10 Bağımsız doğrulama — "peki ya kodunuzda hata varsa?"

Optimizasyon projelerinin en büyük riski şudur: modelde bir hata varsa, çözücü o hatayı **sömürerek** yüksek ama gerçek olmayan bir skor üretir. Kendi kodunuzla kendi kodunuzu kontrol edemezsiniz. Bu yüzden ikinci bir kayıt tuttuk.

10.1 Çift kayıt ilkesi

`05_validate_solution.py`, çözümü **modelden hiç geçmeden** yeniden hesaplar:

- **Pyomo'yu hiç import etmez.** Model kodu bu betikte yüklenmez bile.
- Tüm sabitleri (10 dk ızgara, ±60 dk pencere, 60–300 dk boşluk, α=0,20, τ=45, ±15 dk bant, 10/15 kapasite) **açıkça yeniden yazar**. Model bir sabiti değiştirir de doğrulayıcı güncellenmezse, doğrulama FAIL verir — istenen davranış budur.
- Optimize saatleri modelin değişkenlerinden değil, **çıktı dosyasındaki kaydırma miktarlarından** yeniden türetir.
- Amaç değerini ve baseline'ı sıfırdan, yalnız `derived/` veri katmanından hesaplar.



Tek bir FAIL → çıkış kodu 1 → run_all.py koşuyu durdurur.

05, modelin içinden HİÇ geçmez; sabitleri (Wdev, τ, Xdev, cap) kendi dosyasında yeniden tanımlar.
Böylece modeldeki bir hata, doğrulamada aynı hatayla örtülemez.

Şekil 10.1 — İki bağımsız hesap yolu. Buluşurlarsa sonuç güvenilirdir.

10.2 Sonuç: tüm kontroller PASS

Kontrol	Kapsam	Sonuç
A1 ızgara / pencere / saat aritmetiği	4.554 uçuş hareketi	**PASS**
F kova kapasitesi	7 gün × 10 dk kovalar	**PASS** — aşan kova 0; en dolu kova kalkış 10/10, varış 15/15
A2 rotasyon (gerçek zaman ekseni)	1.709 tur (gece-aşan dahil)	**PASS** — yeni ihlal **0**
G düzenlilik (±15 dk bant)	2.644 gün-arası bağ	**PASS** — bant dışı 0
B1 boşluk / J / saat tutarlılığı	6.966 seçilen bağlantı	**PASS**

Kontrol	Kapsam	Sonuç
C aktarma ödülü, D yenilen rakip + sıralama	4.660 pazar-gün	**PASS**
E1 yön dengesi (α = 0,20)	2.001 yön-çifti	**PASS** — dengesiz 0
Amaç toplamı (bağımsız yeniden hesap)	3.405.039 ↔ 3.405.039	**PASS**
Baseline (bağımsız yeniden hesap)	3.251.527 ↔ 3.251.527	**PASS**

En güçlü satır sondan ikincisi: dosyadaki amaç toplamı ile sıfırdan yapılan yeniden hesap **birebir** tutuyor. Yani raporladığımız 3.405.039 puanın her kuruşu bağımsız olarak yeniden üretilebiliyor.

10.3 Doğrulayıcı gerçekten işe yaradı mı? — evet, üç kez

Bu bir vitrin süsü değil. Gerçek hatalar yakaladı ve hepsi düzeltildi:

1. **Kapasite evreni açığı.** Amaç fonksiyonuna girmeyen (ρ'suz, dışlanan) pazarların uçuşları da IST kovalarını fiziksel olarak dolduruyor, ama model onları saymıyordu. Doğrulayıcı yakaladı; artık o uçuşlar da sabit doluluk olarak sayılıyor.
2. **Rotasyon evreni açığı.** Bir rotasyon ortağının bağlantısı amaç evreninde değilse, o günkü olay hiç modellenmiyordu ve A2 kısıtı kurulamıyordu. Doğrulayıcı **gerçek bir ihlal** yakaladı (uçuş 2200/2201, gün 2): rotasyon ortağının kalkışı kısıtsız kayıp minimum turu 20 dakika altına indirmişti. Düzeltme sonrası skor bilinçli olarak **düştü** (+%4,32 → +%4,27) — haksız kazanılan payı geri verdik.
3. **Kopya olay tutarsızlığı.** Hafta normalizasyonu ihtiyacını (Bölüm 2.4) da bu doğrulayıcı ortaya çıkardı.

> **Jüriye söylenecek cümle**
>
> "Doğrulayıcımız üç gerçek hata yakaladı ve bunlardan biri skorumuzu **düşürdü**. Bunu saklamak yerine belgeledik, çünkü amacımız yüksek bir sayı değil, **doğru** bir sayı üretmek."

Bir dürüstlük ayrıntısı daha: doğrulayıcı, çözücünün bazı "rakip yenildi" göstergelerini bağlamadığını da tespit ediyor (1.199 adet). Bunlar gerçekte yenilen ama raporda sayılmayan rakipler — yani **raporladığımız skor, gerçek skorun bir alt sınırıdır**. Bu yönde hata, skoru şişirmez; tersi olsaydı hata sayılırdı ve o sayaç sıfırdır.

11 Sonuçlar

11.1 Ana sonuç

Ölçüt	Değer
Baseline (mevcut yayınlanan tarife)	3.251.527
Aşama-1 (güne göre ayrıştırma)	3.390.287
Aşama-2 (tam hafta cilalama) — NİHAİ	**3.405.039** (+14.752)
İYİLEŞME	**+153.512 (+%4,72)** gelir-ağırlıklı rekabetçilik
Küresel MIP-gap	**%1,18** — optimalite kanıtı
Kaydırılan uçuş hareketi	1.067 / 4.554 (**%23**) — 3.487'si yerinde kaldı
Toplam kaydırma	20.640 uçuş-dk · ortalama **19 dakika**
Sunulan bağlantı	6.966 / 7.577 aday
Çözülen gün	7 / 7 (G gevşetilen: 0)
Bağımsız doğrulama	Tüm kontroller **PASS** · A2 yeni ihlal: 0

Bu tablodaki en güçlü satır ikili: **%4,72 kazanç, %23 kaydırma.** Yani uçuşların dörtte üçüne hiç dokunmadan, dokunduklarımızı da ortalama 19 dakika oynatarak elde edilmiş bir kazanç. Operasyonel olarak uygulanabilir bir öneri.

11.2 Gün kırılımı

Gün	Pazar-gün	OPT	Baseline	Δ	Kaydırılan
1	670	488.005	465.686	+22.318	166
2	655	465.912	443.873	+22.038	161
3	658	490.860	470.624	+20.237	140
4	661	482.902	459.225	+23.677	155
5	667	495.630	472.018	+23.612	139
6	672	486.872	467.761	+19.111	153
7	677	494.858	472.339	+22.519	153

Kazanç yedi güne dengeli dağılmış — tek bir günü sömürüp ortalamayı yükselten bir çözüm değil. Bu da sonucun sağlamlığının bir göstergesi.

11.3 Duyarlılık — sonuç ne kadar sağlam?

Varyant	Kazanç	Yorum
Referans (±60 dk, τ=45)	+%4,32	aşama-1 ölçümü
Pencere ±30 dk	+%3,64	Pencere serbestliği gerçek bir kaldıraç — beklenen yön
Pencere ±90 dk	+%1,38	**Düşüş serbestlikten değil, süre limitinden**: aday sayısı 13→19'a çıkıyor, 150 s'de çözücü iyi çözüm bulamıyor. Ayrıca bu ayarda doğrulayıcı bir A2 açığı yakaladığı için güvenli bulunmadı ve **iptal edildi**.

Varyant	Kazanç	Yorum
τ = 30 dk	+%4,52	Kısa yer süresi → rotasyon gevşer, kazanç artar
τ = 60 dk	+%4,43	Dar bantta; monoton değil (gerekçesi raporda)

Duyarlılık analizinin en dürüst kısmı üçüncü satır: sonucun düşmesinin sebebini "model kötüleşti" diye değil, **"ölçüm zaman-limitliydi"** diye açıkladık ve o varyantı benimsemedik.

12 Jüri soru bankası — bu dokümandaki konular için

Aşağıdaki sorular tam olarak bu dokümanın kapsadığı alanlardan. Cevapları ezberlemeyin; mantığını anlayın, kendi cümlelerinizle söyleyin.

S1 MIP gap %1,18 diyorsunuz. Bu sayıyı siz mi hesapladınız?

"Hayır, çözücü hesaplıyor; biz yalnız okuyoruz. HiGHS dallanma-sınırlama sırasında iki sayı tutar: elindeki en iyi gerçek çözüm (incumbent) ve LP gevşetmelerinden gelen üst sınır (bound). Kodumuzun 941. satırı bu iki değeri çözücünün sonuç nesnesinden okuyor, 943. satır farkı incumbent'a bölüyor. Yani tek yaptığımız bir bölme işlemi — sınırın kendisi çözücünün matematiksel çıktısı."

S2 Peki bu sınırın geçerli olduğunu nereden biliyoruz?

"Doğrusal programlamanın temel bir sonucu: bir MIP'in LP gevşetmesi, tamsayı kısıtı kaldırıldığı için daha geniş bir fizibıl bölgeye sahiptir. Enbüyüklemede daha geniş bölgenin en iyisi, dar bölgenin en iyisinden küçük olamaz. Dolayısıyla her düğümdeki LP değeri o dalın üst sınırıdır; hepsinin en büyüğü de global üst sınırdır. Bu bir varsayım değil, teorem."

S3 Gap hangi amaç fonksiyonu üzerinde? Raporladığınız %4,72 üzerinde mi?

"Hayır, ve bunu raporlarımızda açıkça etiketledik. Gap, çözücünün optimize ettiği **cezalı** amaç üzerinde: rekabetçilik eksi λ çarpı toplam kaydırma. Raporladığımız +%4,72 ise **cezasız** saf rekabetçilik — çünkü yarışmanın ölçütü o. Ceza terimi bizim eklediğimiz operasyonel gerçekçilik katmanı; ödülü şişirmiyor, yalnız hangi kaydırmaların yapılacağını süzüyor."

S4 Neden sezgisel bir yöntem (genetik algoritma) kullanmadınız?

"Çünkü sezgisel yöntemler çözümün kalitesi hakkında hiçbir garanti veremez. Bir genetik algoritma size bir tarife verir ama optimumun %2 mi %40 mı altında olduğunuzu söyleyemez. MIP ise bir üst sınır üretir. Bizim için ölçülebilir güven, biraz daha yüksek olabilecek ama kanıtsız bir skordan daha değerliydi. Nitekim aşama-1'imiz greedy bir sezgiseldir ve tam da bu yüzden yetmedi: aşama-2'yi eklememizin sebeplerinden biri kanıtın kendisiydi."

S5 E2 kısıtını kapatmışsınız. Bu kuralı ihlal etmek değil mi?

"E2 formüle edildi, kodda kurulu ve küçük ölçekte doğrulandı — yapamadığımız bir şey değil. Tam ölçekte kapatmamızın iki ölçülmüş sebebi var. Birincisi: E2'nin min-seçimi büyük-M'i LP gevşetmesini o kadar zayıflatıyor ki çözücü **hiç uygun çözüm bulamıyor**; bu bir boyut sorunu da değil — tek gün bile çözülemiyor, aynı gün E2 kapalıyken saniyeler sürüyor. İkincisi: E2-kapalı optimal çözümde yön çiftlerinin **%76'sı** 30 dakikalık eşiği zaten ihlal ediyor, en büyük fark 465 dakika. Çünkü bu farkın çoğu bacak uçuş sürelerinden geliyor ve onlar sabit veri — hiçbir tarife düzenlemesiyle kapatılamaz. Kapatma kararı model dokümanında, teknik raporda ve çıktı özetinde açıkça yazılı."

S6 Kaç ikili değişkeniniz var? Belgelerde iki farklı sayı görüyorum.

"Teslim ettiğimiz tam-hafta modelinde **100.552**. 100.566 ise aynı modelin monolitik varyantı — orada bir olay, gün-görünümü kopyası olduğu için aşama-1 değerine sabitleniyor ve 12 zaman ataması ile 2 yenme göstergesi düşüyor. İkisi de doğru; farkı yeniden sayarak doğruladık ve dökümü Ek A'da veriyoruz."

S7 Modelinizde bir hata olmadığını nereden biliyorsunuz?

"Bilmiyoruz — bu yüzden ikinci bir kayıt tuttuk. `05_validate_solution.py` çözümü model kodundan hiç geçmeden, Pyomo'yu import bile etmeden, yalnız veri katmanından yeniden hesaplıyor. Tüm sabitleri bağımsız olarak yeniden yazıyor. Ve gerçekten işe yaradı: üç gerçek hata yakaladı, bunlardan biri skorumuzu %4,32'den %4,27'ye **düşürdü**. Haksız kazanılmış puanı geri verdik."

S8 Sonuç çözücünün rastgeleliğine bağlı mı? Tekrar üretilebilir mi?

"Tam tekrar üretilebilir. Çözücüyü `threads=1` ve `random_seed=0` ile çağırıyoruz; paralel iş parçacığı yok, sabit tohum var. Aynı komut aynı makinede aynı sonucu verir. Gün listesi bile veriden okunuyor, koda gömülü değil — gizli test verisi farklı gün etiketleriyle gelirse kod çalışmaya devam eder."

S9 Neden ±60 dakika? Neden 10 dakikalık ızgara?

"10 dakika, brief'in hub kapasite kısıtının zaten tanımlı olduğu kova genişliği — aynı ızgarayı kullanmak kısıtı doğal kılıyor. ±60 dakika ise bizim kararımız: brief sınır koymuyor ama sınırsız bırakmak hem modeli patlatır hem operasyonel olarak anlamsız olur. Duyarlılık analizinde ±30 ve ±90 da denendi; ±30 kazancı düşürüyor, ±90 ise problem büyüdüğü için verilen sürede iyi çözüm bulunamıyor ve ayrıca doğrulayıcı orada bir açık yakaladığı için benimsemedik."

S10 Ceza terimi eklemek kazancı artırmış. Bu nasıl olabilir?

"Dejenerasyon. Cezasız modelde binlerce farklı tarife tamamen aynı amaç değerini veriyor — çünkü fayda üretmeyen bir kaydırma bedava. Bu simetri LP gevşetmesini gevşetiyor ve çözücü iyi bir tamsayı çözümü bulamıyor. Ceza her olay için tek bir 'minimum sapma' yönü tanımlayıp simetriyi kırıyor, gevşetmeyi sıkıyor. Sonuç: kaydırma %92'den %25'e inerken kazanç %3,89'dan %4,32'ye çıktı. λ'yı 0,1 ile 1,5 arasında taradık; sonuçların dar bir bantta kümelenmesi de gerçek optimuma yakınlığın işareti."

A Ek A — Sayı künyesi ve yeniden sayım kanıtı

A.1 Girdiden çözüme tek tabloda

Kategori	Ölçüt	Değer
Girdi	Ham Excel satırı (4 dosya)	89.094
	Bağlantı tablosu satırı	57.317 (TK 7.850 · rakip 49.467)
	Operasyon günü	7 *(veriden okunur, koda gömülü değil)*
	Yönlü O–D pazarı	1.408 (p'suz olduğu için dışlanan: 48)
	TK uçuş numarası	952 (eşli 776 · partnersiz 176)
Türetilmiş	TK bağlantı adayı	7.693
Model	Pazar-gün grubu	4.660
	Aday bağlantı (π)	7.577
	Uçuş olayı	5.376 (ayarlanabilir 4.553 · sabit 823)
	İkili değişken (teslim modeli)	**100.552**
	İkili değişken (monolitik varyant)	100.566
Çözüm	Baseline	3.251.527
	Nihai amaç (saf rekabetçilik)	**3.405.039**
	İyileşme	**+153.512 (+%4,72)**
	Global MIP-gap (cezalı amaç)	**%1,183**
	Çözüm süresi	1.809 s + aşama-1 ~1,5 dk

A.2 İkili değişken sayımı — iki modelin karşılaştırması

Aşağıdaki sayılar, her iki modeli yeniden kurup Pyomo üzerinden **tek tek sayarak** üretildi (çözüm yapılmadan, hiçbir çıktı dosyasına dokunulmadan). Belgelerdeki iki farklı toplamın kaynağı budur:

Değişken ailesi	Teslim modeli (08)	Monolitik varyant (03 full)	Fark
x — zaman ataması	**59.177**	59.189	−12
γ — yenilen sayı seçici	11.690	11.690	—
y — bağlantı sunumu	7.577	7.577	—
z — azalan getiri slotu	7.577	7.577	—
w — koşullu yenme	**7.529**	7.531	−2
β — rakip yenildi mi	7.002	7.002	—
TOPLAM	**100.552**	**100.566**	**−14**
Kısıt sayısı	67.877	70.359	
G bağı	2.643 (ilk-gün çapalı)	4.503 (ortak bant)	
A2 bağı · F bağı	874 · 1.395	874 · 1.395	

Farkın açıklaması: Teslim modelinde bir olay, aynı fiziksel zaman damgasının birden çok operasyon gününde kayıtlı olması nedeniyle (gün-görünümü kopyası) aşama-1 değerine sabitlenir. O olayın 13 aday saati 1'e iner (−12 `x`) ve ona bağlı iki koşullu yenme göstergesi gereksizleşir (−2 `w`). Kısıt sayısındaki fark ise G'nin iki modelde farklı kurulmasından gelir: teslim modeli, bağımsız doğrulayıcıyla birebir aynı "ilk-gün çapalı" biçimi kullanır.

Belgelerdeki tutarsızlığın giderilmesi

`reports/fullweek_polish.md`, teknik rapor ve model dokümanı **100.552** diyor (teslim modeli — doğru). Kod Rehberi ve sunum figürü **100.566** diyor (monolitik varyant — o da doğru, ama teslim edilen model değil). Sunumda tek bir sayı kullanacaksanız **100.552**'yi tercih edin ve sorulursa yukarıdaki farkı açıklayın.

A.3 Nereye bakmalı — dosya haritası

Aradığınız şey	Dosya
Matematiksel formülasyon (kümeler, kısıtlar, amaç)	`docs/model_formulation.md`
Formülasyon gerekçeleri, varsayımlar, ölçekleme	`reports/technical_report.md`
Kodun blok blok anlatımı	`docs/Buke_KOD_REHBERI.docx`
MIP gap koşu raporu	`reports/fullweek_polish.md`
Bağımsız doğrulama sonuçları	`reports/validation_report.md`
Duyarlılık analizi	`reports/sensitivity_analysis.md`
Veri sağlık teşhisi	`reports/data_health_report.md`
Nihai tarife çıktısı	`output/optimized_schedule.csv`
Pazar bazında sonuçlar	`output/market_results.csv`
Modelin kendisi	`src/03_model.py` (kısıtlar 372–748, çözücü 917–948)
Tam hafta cilalama	`src/08_fullweek_polish.py`
Bağımsız doğrulayıcı	`src/05_validate_solution.py`

A.4 Belgelerde fark ettiğimiz küçük tutarsızlıklar

Sunum öncesi bilmenizde fayda var; jüri kodu okursa karşılarına çıkabilir:

- **İkili değişken sayısı** (100.552 / 100.566) — A.2'de açıklandı. Gerçek bir çelişki değil, iki farklı model.
- **`03_model.py` satır 101'deki yorum** "appsi Highs MIP-start'ı kullanmıyor — etkisiz" diyor. Bu yorum **eskimiş**: kurulu Pyomo sürümünde sıcak başlangıç gerçekten HiGHS'e aktarılıyor ve aşama-2 tam da buna dayanıyor. Kod doğru çalışıyor, yalnız yorum güncellenmemiş. Sorulursa: "*O yorum eski bir Pyomo sürümünden kalma; aşama-2'nin sıcak başlangıcı fiilen çalışıyor ve kazancı raporda ölçülü.*"
- **Olay sayısı** kaynağa göre 5.376 (model evreni) veya 5.473 (doğrulayıcı evreni) görünebilir. Fark beklenen: doğrulayıcı, amaç fonksiyonuna girmeyen pazarların uçuşlarını da sayar — çünkü onlar da hub kapasitesini fiziksel olarak dolduruyor.
- **MIP gap figürü** (Şekil 9.1) şematiktir: başlangıç değeri, nihai değer ve %1,18'lik boşluk gerçek koşudan, aradaki yakınsama eğrileri temsilîdir. Sorulursa bunu peşinen söyleyin.

Son söz

Bu projenin jüri karşısındaki en güçlü yanı yüksek bir skor değil — **her sayının nereden geldiğini gösterebiliyor**

olmanız. Kaç değişken var, neden o kadar; hangi kısıt neden kapalı, hangi ölçümle; skor ne kadar iyi, hangi kanıtla; ve kod hata yaparsa bunu kim yakalıyor. Sunumda bu dört soruya rahatça cevap verebiliyorsanız, teknik derinlik zaten görünür olur.